Sentinel Platform

Unified CCTV Integration & AI Video Analytics
High-Level Design and supporting documents

Gujarat Police Innovation Challenge 2026
Generated 2026-10-01

Contents

1. 00 — SOLUTION PLAN
2. 01 — ARCHITECTURE
3. 02 — ANPR PIPELINE
4. 03 — UX DESIGN
5. 04 — SCALE PLAN
6. 05 — DELIVERY PLAN
7. 06 — SUBMISSION PLAYBOOK
8. 07 — DRIVER SDK
9. 08 — SECURITY HARDENING
10. 09 — SECONDARY ANALYTICS
11. 12 — REQUIREMENTS TRACEABILITY

Sentinel Platform — Master Solution Plan

Gujarat Police Innovation Challenge 2026 — Unified CCTV Integration & Video Analytics Platform

> **Planning-figure correction (M15).** Figures of "~50 cameras" and references to a `GET /api/ingest` catalogue endpoint in this document date from planning, before the grid was accessible. The real grid assigned to us has **30 cameras**, and our host serves its catalogue at **`GET /cameras.json`** behind a session-cookie login — the integrator guide's `/api/ingest` path 404s there, because that guide is a shared template with `<host>` placeholders. The planning text is left as written rather than quietly rewritten, so the record of what was assumed stays visible; where this document states a present-tense fact, trust the README and `evidence/`.

Product name	**Sentinel Platform**. ⚠️ *Note: this is also the organisers' programme name. In the PPT and HLD, always write "the Sentinel Platform" for the product and "the Challenge" / "the government test grid" for the organisers' assets, so a judge is never confused about what is yours.*
Entrant	Solo individual, registering under **Category 1 — Student / Researcher / Professional**
Chosen architecture	**Hybrid: Model 1 + Model 2 + Model 3 + selective Model 4** (registry/GIS + unified viewing + vendor federation + event-driven central recording)
Target	Win — build for Phase-2 production credibility from day one
Demo hardware	Apple Silicon M4 (local), M2 (secondary). Colab GPU for any fine-tuning. No AWS at demo time; AWS/K8s is the *documented* production path.
Delivery surfaces	Operator Console (desktop web) + Field PWA (mobile) + Admin Portal — one design system, three shells
Source code	**Private repository.** Submitted as a hosted archive/private link, not a public GitHub URL. See §9.
Face recognition	**Out of scope by decision.** Vehicle-centric analytics only; the privacy position is argued positively. See §6.
Documents	This master plan → 01-ARCHITECTURE · 02-ANPR-PIPELINE · 03-UX-DESIGN · 04-SCALE-PLAN · 05-DELIVERY-PLAN · 06-SUBMISSION-PLAYBOOK

---

1. The strategic read — what actually wins this

This is not judged as a hackathon demo. It is judged as a **procurement shortlist**. Seven scored areas, all mandatory, and bonus points only count once every mandatory box is ticked. So the plan is built backwards from the rubric.

#	Scored area	What the jury is really asking	How Sentinel answers it
1	Successful Test Case	"Can you onboard our heterogeneous feeds and *actually* analyse them?"	Catalogue-driven auto-onboarding from the grid's catalogue endpoint (`cameras.json` on our assigned host — the guide's generic `/api/ingest` 404s there); all 30 cameras live in the registry in one call; continuous ANPR on the analytics-enabled subset with tiered scheduling
2	Solution Presentation	"Do you understand our problem, or did you bring a generic VMS?"	Model justification grounded in *their* four challenges + bandwidth/cost math that proves why full centralisation fails
3	Solution Architecture	"Is this deployable in a government network without a rip-and-replace?"	Edge-first, vendor-neutral adapter framework, ONVIF-first, mTLS, department tenancy, zero changes to existing dept VMS
4	Working Platform + Demo	"Is this real software or a Figma file?"	One-command `docker compose up`, real backend, real DB, real streams, no mocked UI states; source archive + guided code tour supplied to reviewers
5	Video Analytics Output	"Is your ANPR good enough to trust on evidence?"	Multi-stage ANPR + temporal voting + confidence scoring + exportable evidence report (CSV/PDF with snapshots, plate, camera, GPS, PTS-accurate timestamp)
6	Scalability + PoC Readiness	"Show me the 80,000-camera number, not the word 'scalable'."	Quantified tiering, GPU sizing, bandwidth math, storage tiers, phased district rollout, capex/opex in ₹
7	Submission Completeness	"Is everything accessible and consistent?"	Submission playbook with link-verification in incognito, consistent naming across PPT/HLD/videos/repo

Bonus magnets we deliberately build (the official bonus list, one-to-one): cross-camera vehicle re-identification + GIS route reconstruction · working alert prioritisation with triage workflow · analytics beyond ANPR (vehicle class/colour, person, intrusion/loitering, camera-tamper) · low-bandwidth metadata-only degradation mode · privacy masking, hash-chained audit trail, RBAC/ABAC · camera health & ops dashboard · integration-ready REST APIs with contract stubs for VAHAN / SARTHI / eGujCop / AFIS / NAFIS.

1.1 The three moments that win the room

Everything else is table stakes. Rehearse these three:

1. **The BOLO moment.** Judge reads out a vehicle number on evaluation day. You type it into one box. The system **simultaneously** (a) retro-searches every event ever ingested and (b) arms a forward-looking watchlist rule. Within seconds the map draws the vehicle's timestamped route across cameras, with snapshot evidence at each hop, and new sightings start landing live as alerts. One input, two directions in time.
2. **The unreadable-plate moment.** A plate the OCR can't fully resolve. You show fuzzy/partial matching (`GJ01?B12?4`), OCR-ambiguity classes (0↔O, 1↔I, 8↔B, 5↔S, 2↔Z), and appearance-based vehicle re-identification bridging the gap — then a spatio-temporal feasibility check that rejects physically impossible hops. This is the difference between a toy and a forensic tool.
3. **The honesty moment.** A slide that says: *"Centralising 80,000 cameras costs 160 Gbps of backbone and 12.1 PB of hot storage for a single week. We do not propose that. Here is the edge-first design that runs on ~140 Mbps — about 1/1,100th of the bandwidth — and here is exactly where the video does and does not travel."* Juries fund people who show them the number they were afraid to ask about.

---

2. Architecture decision — why this hybrid

> **Rule from the organisers:** Model 1 is mandatory for everyone and insufficient alone. Hybrid is explicitly allowed and bonus-worthy.

Sentinel = Model 1 (foundation) + Model 2 (default integration path) + Model 3 (federation where a VMS exists) + Model 4-selective (event-driven central recording, not bulk archival).

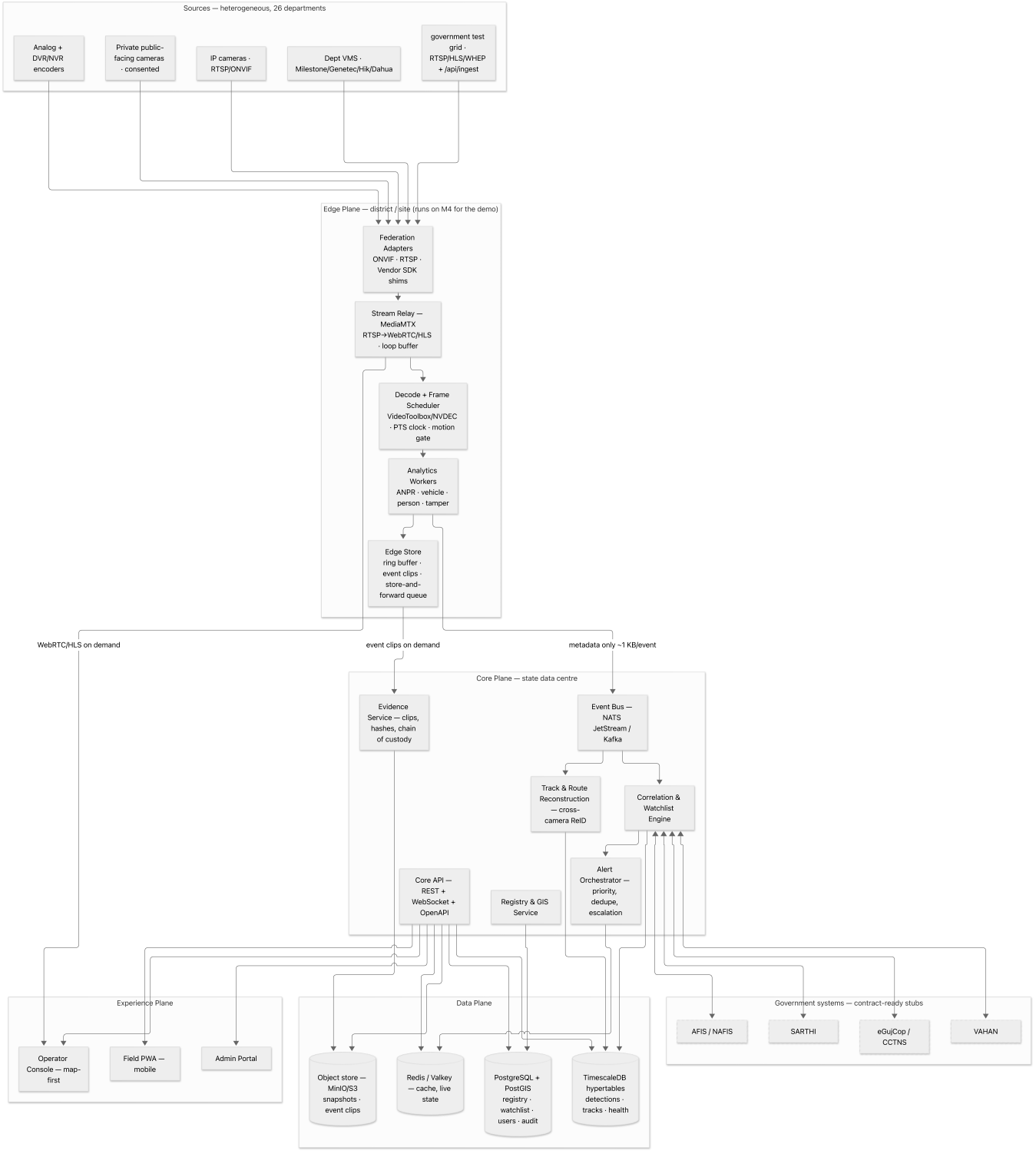
Model	How we implement it	Why
1 — Registry & GIS *(mandatory)*	Central camera inventory, PostGIS geometry, MapLibre GIS console, bulk CSV + manual + API onboarding, health monitoring, coverage gap analysis, dept-wise RBAC, full audit	Mandatory, and it is the cheapest high-scoring component. It is also the *system of record* every other subsystem reads from.
2 — Unified viewing + metadata analytics	Direct RTSP/ONVIF pull → local relay → WebRTC/HLS to browser. Dept systems untouched. Metadata (not video) flows to the centre.	This is the realistic path for the ~50 test feeds and for the majority of the 80k. Lowest infra, highest demo reliability.
3 — VMS federation middleware	Adapter/plugin framework with a stable `CameraSource` contract: ONVIF Profile-S/T, generic RTSP, and stub-but-real adapters for Milestone / Genetec / Hikvision / Dahua patterns. One normalised downstream API.	This is what actually unblocks 26 departments. Working adapter *framework* + 2 live adapters proves extensibility without needing 26 vendor licences.
4 — Central VMS *(selective)*	**Event-driven recording**: continuous short-loop buffer at the edge; on an event/alert, the pre/post-roll clip is sealed, hashed, and pushed to central object storage as forensic evidence. Plus continuous recording for a **designated small set** of high-value cameras, with timeline scrubbing playback.	Gives you the full Model-4 user experience (record, playback, scrub, export, chain-of-custody) and the DR/tiering story, **without** claiming a 12 PB/week archive you cannot build. This is the defensible version of "we do Model 4 too".

Architecture Decision Records (full ADRs with rejected alternatives live in 01-ARCHITECTURE §2):

- **ADR-001** Edge-first analytics, metadata-to-centre — *rejected:* central decode of all streams (160 Gbps).
- **ADR-002** Event-driven + selective continuous recording — *rejected:* statewide 24×7 central archive (12.1 PB/7-day hot tier).
- **ADR-003** Adapter/plugin federation over per-vendor forks — *rejected:* one monolith per VMS vendor.
- **ADR-004** Python everywhere for services, TypeScript for UI, off-the-shelf Go binaries (MediaMTX, NATS) we never fork — *rejected:* polyglot Go/Java services (solo dev, no second language budget).
- **ADR-005** PTS-derived time as the single source of truth, never arrival time — mandated by the organisers' integrator guide and scored.
- **ADR-006** PostgreSQL + PostGIS + TimescaleDB as one primary store; OpenSearch deferred to the scale plan — *rejected:* Elasticsearch on day one (solo ops cost, no accuracy benefit at demo scale).
- **ADR-007** Mobile as an installable PWA, not React Native — *rejected:* RN (build/signing overhead for a solo entrant; PWA is demo-safe and offline-capable).

---

3. System shape (one picture)



The load-bearing idea: *video stays local, meaning travels.* An ANPR event is ~1 KB of JSON plus a ~30 KB snapshot. A 1080p stream is 2 Mbps. Everything in the design follows from that ratio.

---

4. Technology stack (final)

Chosen for: open-source mandate · Apple Silicon demo · one-person maintainability · a straight line to the AWS/K8s production story.

Layer	Choice	Licence	Why this, not the obvious alternative
Edge/relay	**MediaMTX**	MIT	One Go binary: RTSP in → WebRTC(WHEP)/HLS/RTSP out, plus recording & playback server. Gives us Model-2 viewing *and* Model-4 recording for free. Beats hand-rolled Janus/Ampache plumbing.
Decode	**FFmpeg / PyAV** with VideoToolbox (Mac) → NVDEC/DeepStream (prod)	LGPL/BSD	Hardware decode is the only way ~50 streams fit on one M4.
AI runtime	**ONNX Runtime** (CoreML EP on Mac, TensorRT/CUDA EP in prod)	MIT	One model artefact, three accelerators. Avoids a PyTorch-on-MPS-only dead end.
ANPR	`fast-alpr` + `open-image-models` + `fast-plate-ocr` (all MIT); Ultralytics YOLO (AGPL) for vehicles; vendored MIT ByteTrack — see 02-ANPR-PIPELINE	MIT / AGPL-3.0	Multi-stage detector → tracker → plate → rectify → OCR → temporal vote. Licence position stated openly, with an Apache-only migration path for Phase 2.
Services	**Python 3.13 + FastAPI + asyncio/uvloop**, Pydantic v2	MIT/BSD	Single language for a solo dev; the CV ecosystem is Python. Throughput risk is mitigated by process-per-camera-group workers and an off-the-shelf broker.
Event bus	**NATS JetStream** (Kafka-compatible abstraction retained)	Apache-2.0	Single 20 MB binary, runs on the M4, real persistence/replay. Kafka is the documented production swap; we keep a thin `EventBus` port so it is a config change.
Primary DB	**PostgreSQL 17 + PostGIS + TimescaleDB**	PostgreSQL/Apache-2.0	Registry geometry, time-series detections, and fuzzy plate search (`pg_trgm`) in one engine.
Search	**Postgres FTS + pg_trgm** now; **OpenSearch** in the scale plan	—	At demo scale, Postgres wins on ops cost with identical UX. Honest, and documented.
Object store	**MinIO** (local) → S3 (prod)	AGPL-3.0 / —	S3 API parity means zero code change between demo and production.
Cache/live state	**Valkey** (Redis fork)	BSD	Live camera state, alert dedupe windows, rate limits.
Frontend	**React 19 + TypeScript + Vite + Tailwind v4 + shadcn/ui (Radix) + Motion**	MIT	Accessible primitives + full styling control = the only realistic path to Apple-level polish for a backend-heavy solo dev.
Maps	**MapLibre GL JS + deck.gl**, OSM / Bhuvan tiles, PMTiles for offline	BSD/MIT	Vector tiles, 60 fps, genuinely beautiful. Leaflet is the stated hint but MapLibre is its open successor — note this in the PPT.
Video in browser	**hls.js** + native **WebRTC WHEP** client	Apache-2.0	WHEP for the low-latency wall, HLS for firewalled networks and mobile.
AuthN/Z	**Keycloak** (OIDC) or self-hosted JWT + Casbin-style policy	Apache-2.0	Department tenancy, RBAC + ABAC, SSO story for government.
Orchestration	**Docker Compose** (demo) → **Kubernetes/Helm** manifests (prod story)	Apache-2.0	`docker compose up` must work first try on a judge's machine.
Observability	**OpenTelemetry + Prometheus + Grafana**, structured JSON logs	Apache-2.0	Camera-health and pipeline-lag dashboards are explicitly bonus-scored.

What we deliberately do not build: our own VMS recorder, our own tracker, our own OCR architecture, our own map renderer, our own auth. Every one of those is a solved open-source problem, and the jury scores integration judgement, not NIH.

---

5. Non-negotiable compliance with the organisers' integrator guide

These are scored directly via the official pre-submission checklist, and most failures come from ignoring them. They are baked into the ingest layer as enforced defaults, not left to discipline:

Rule	Implementation
Force RTSP over TCP	`rtsp_transport=tcp` hard-coded in every capture path; HLS auto-fallback if 8554 is blocked. Covered by a unit test that asserts the option is set.
All timing from PTS, never arrival	A `StreamClock` type wraps every frame. Arrival time is **not exposed** to analytics code. Trackers/Kalman consume PTS deltas only.
Exponential backoff reconnect	Supervisor with 2 s → 30 s jittered backoff, per-camera circuit breaker, no tight loops.
Catalogue-driven discovery	Every run starts from `GET /api/ingest`; camera IDs are never hard-coded; a reconciliation job diffs catalogue vs registry and auto-onboards changes.
Mixed H.264/H.265, mixed resolution	Per-camera decoder config from catalogue properties; **ragged batching** (group by resolution class), never a fixed-shape batch.
Decode warnings at join are not fatal	Decoder errors are logged at `debug` until the first IDR; only sustained failure trips the breaker.
Don't trust `CAP_PROP_FPS`	Effective frame rate is *measured* from PTS in a rolling window; the declared value is recorded as metadata only.
Tolerate inter-frame gaps	Gap ≠ disconnect. Motion models use elapsed PTS; the breaker trips on stall duration, not on a missing frame.
Survive the loop cut (scene discontinuity)	Loop/cut detector (histogram + PTS regression) → resets background models, ReID galleries and track IDs cleanly, and emits a `stream.discontinuity` event that shows up in the health dashboard.
Consume only; never publish or call control API	Relay config is pull-only; publishing endpoints are disabled in config and asserted at startup.
Open only cameras you process	Lazy capture with reference counting; idle captures closed after a TTL. A live "open captures" counter is on the ops dashboard.
No reliance on downloaded footage	Everything is built against live capture from day one; `/stream/<id>` is never fetched as a file.

> Turning this checklist into a visible **"Integrator Compliance" panel** in the ops dashboard (green ticks, live counters for backoff events, discontinuities, measured vs declared FPS) is a cheap and very deliberate signal to a technical jury that you read their guide.

---

6. Feature scope — MoSCoW

Must (no submission without these)

- Catalogue-driven onboarding of all ~50 test cameras; bulk CSV + manual + API onboarding for the general case
- PostGIS registry + GIS map with department/type/status layers and coverage gap analysis
- Unified live viewing wall (WebRTC primary, HLS fallback) with per-tile health
- Continuous ANPR with PTS-accurate timestamps and snapshot evidence
- Watchlist DB (stolen / wanted / missing / suspect / BOLO) + continuous correlation + real-time alerts
- Vehicle search → timestamped, location-wise route on GIS + event timeline
- Alert triage workflow (acknowledge / assign / resolve / mark false positive)
- Camera health monitoring + ops dashboard
- RBAC by department + immutable audit trail
- Evidence report export (CSV + PDF) for the govt-feed demo
- One-command deploy; OpenAPI docs; reviewable source archive

Should (these are the bonus points)

- Cross-camera vehicle ReID + spatio-temporal route validation
- Event-driven central recording + timeline playback + clip export with hash chain-of-custody
- Adapter framework with ≥2 working adapters + documented plugin SDK
- Alert prioritisation/scoring, dedupe, escalation rules
- Low-bandwidth degradation mode (metadata-only, snapshot downscale, store-and-forward)

- Privacy: bystander/face blurring by default, reveal-on-authorisation with reason capture
- Mock-but-contract-real VAHAN/SARTHI/eGujCop/AFIS/NAFIS connectors behind a provider interface
- Field PWA with push alerts and offline queue

Could

- Additional analytics: helmet/triple-riding, wrong-way, intrusion/loitering, crowd density, abandoned object, camera tamper
- Natural-language search ("red hatchback near Sector 18 yesterday evening")
- Automatic gap-analysis recommendations for new camera placement

Won't (this cycle — and say so out loud, it builds credibility)

- Real integration with production VAHAN/AFIS (no access granted; we ship the contract + a conformant mock)
- **Face recognition of any kind** — a deliberate scope decision, not a gap. Faces are *detected only in order to be blurred*. The system is vehicle-centric: it answers "where did this vehicle go", not "where did this person go". Say this proactively; it is the strongest privacy position available and it pre-empts the single most common criticism of statewide CCTV programmes.
- Statewide 24×7 central archive (deliberately rejected — see the cost model)

---

7. What each deliverable becomes

Required submission	Sentinel artefact
(1) Solution Presentation	`deliverables/presentation/` — model justification, architecture, analytics, alerting, scale, policing impact
(2) Technical Proposal / HLD	01-ARCHITECTURE + 04-SCALE-PLAN exported to PDF
(3) Demo on own feed (2–3 min)	Scripted run: onboard a self-hosted feed → live ANPR → watchlist hit → alert → GIS route. Storyboard in 06-SUBMISSION-PLAYBOOK
(4) Demo on govt feed + output report	Overnight run across the ~50-camera grid → auto-generated plate/timestamp evidence report + screen recording
(5) Scale plan to ~80k	04-SCALE-PLAN — bandwidth, GPU, storage, HA/DR, security, ₹ capex/opex, phased rollout

---

8. Risk register

Risk	Impact	Mitigation
M4 cannot sustain ~50 concurrent decodes + inference	Demo stalls — fatal	Tiered analytics scheduler (full-rate tier / sampled tier / registry-only tier), motion gating, hardware decode, downscaled decode, measured capacity documented honestly. Capacity benchmark is **Milestone 1**, before any feature work.
ANPR accuracy poor on Indian plates at CCTV angles	Kills scored area 5	Multi-stage pipeline with rectification + temporal voting across a track; fuzzy/partial matching so a partially-read plate still finds the vehicle. **No credible pretrained Indian weights exist publicly — budget 1–2 Colab fine-tuning days up front rather than treating it as contingent.**
Test grid camera IDs/props change before evaluation	Silent breakage	Catalogue-driven by contract; nightly reconciliation; zero hard-coded IDs; integration test that boots from a catalogue fixture
Scope explosion (3 UI surfaces, solo)	Nothing finishes well	Single design system, one codebase, three shells; Should/Could features strictly gated behind Must completion; weekly demo-able increments
Losing time to infra yak-shaving	Lost days	Off-the-shelf binaries (MediaMTX, NATS, Postgres, MinIO) via Compose; no custom infra
Privacy/ethics challenge from jury	Credibility hit	Privacy-by-design section in the HLD: blurring, purpose limitation, retention policy engine, four-eyes reveal, false-match redressal, DPDP Act 2023 alignment
Demo-day network blocks RTSP/8554	Demo dies live	HLS fallback path tested and rehearsed; a recorded fallback video kept ready

---

9. Decisions taken

#	Decision	Consequence for the build
1	**Category 1 — Student / Researcher / Professional**	Phase-1 pool ₹7 L; jury weights innovation and feasibility over infra muscle. No DPIIT certificate needed.
2	**Product name: "Sentinel Platform"**	Shares a name with the organisers' programme — so every document must consistently say *"the Sentinel Platform"* for the product and *"the Challenge"* / *"the government test grid"* for their assets. A cover slide reading "Sentinel Platform — submitted to the Sentinel Challenge" removes all ambiguity in one line.
3	**Private repository, permanently**	Submit a hosted source archive or a private link with reviewer access instead of a public GitHub URL. See the mitigation below — this one costs you points unless it is handled deliberately.
4	**No face recognition**	Simplifies the analytics plane and the privacy story. Faces are detected *only to blur them*. Milestone M13 drops FRS; the freed time goes to vehicle attributes and zone analytics.

Mitigating the private-repo decision

"Working Platform + Demo" is a scored area, and an open repository is a cheap, high-trust proof that working software exists. Keeping it private is legitimate, but the evidence has to come from somewhere else. Compensate deliberately:

- **Show the terminal on camera, repeatedly.** `docker compose ps`, live logs scrolling, `pytest` running green, an API call returning real JSON. Backend liveness has to be *visible* because it cannot be *inspected*.
- **Ship a source archive with the submission** (zip, or a private link with viewer access for the evaluators). The challenge asks for a repo link *optionally*; an archive plus a clear file tree satisfies the same intent.
- **Publish the artefacts, not the code:** the OpenAPI specification, the architecture docs, the accuracy benchmark CSVs, the capacity curves. These prove rigour without exposing implementation.
- **Include a `SOURCE-TOUR.md`** in the archive — a five-minute guided reading path through the codebase for a reviewer with limited time. Most private submissions are never actually read; a guided tour changes that.
- **Reconsider at submission time.** If you later decide the openness is worth more than the protection, flipping a private repo public takes one click, and the challenge explicitly mandates open-source technologies — a jury may read a closed repo as being in tension with that spirit. Worth one deliberate re-think before you upload.

Next: read 01-ARCHITECTURE for component-level design and ADRs.

01 — Architecture & High-Level Design

Sentinel · Unified CCTV Integration & Video Analytics Platform

> This document doubles as the **Technical Proposal (HLD)** deliverable. Export to PDF at submission time.

> **Planning-figure correction (M15).** Figures of "~50 cameras" and references to a `GET /api/ingest` catalogue endpoint in this document date from planning, before the grid was accessible. The real grid assigned to us has **30 cameras**, and our host serves its catalogue at `GET /cameras.json` behind a session-cookie login — the integrator guide's `/api/ingest` path 404s there, because that guide is a shared template with `<host>` placeholders. The planning text is left as written rather than quietly rewritten, so the record of what was assumed stays visible; where this document states a present-tense fact, trust the README and `evidence/`.

> **Implementation status (M3, this session):** Model 1 (the mandatory registry + GIS foundation) is built and verified against a real PostGIS-backed Postgres — not just designed. See `services/core_api/src/core_api/registry/` (`service.py`, `bulk_import.py`, `gap_analysis.py`) and `services/core_api/src/core_api/db/models.py`. Async SQLAlchemy 2.0 + GeoAlchemy2 + Alembic, with real GIST-indexed geometry columns for camera location and FOV. All ~8 synthetic-grid cameras (a proxy for the ~50-camera government grid) auto-onboard via catalogue-driven upsert and are idempotently re-onboardable (a second sync updates in place, never duplicates). Coverage gap analysis is a genuine PostGIS `ST_SquareGrid` + `ST_Distance` query, not a mocked calculation — confirmed to correctly report both "covered" and "uncovered" outcomes depending on the configured radius. 34 integration tests (skipped, not failed, when Postgres isn't running — see `services/core_api/tests/conftest.py`) plus the full unit-test tier, all green. Two real async-SQLAlchemy bugs were found and fixed while building this: `pool_pre_ping=True` triggers a documented greenlet-context error with the asyncpg driver, and an ORM object returned from a write path must have every relationship it will be read through explicitly refreshed — async sessions do not support implicit lazy-loading.

---

1. Design principles

1. **Video stays local, meaning travels.** Bandwidth and storage economics dictate the entire topology. A detection event is ~1 KB; a stream is 2 Mbps. Ratio ≈ 1:2000.
2. **Never modify a department's existing system.** Read-only, pull-based, standards-first (ONVIF → RTSP → vendor SDK, in that order of preference). No agent installed on dept infrastructure unless the department asks for one.
3. **Everything is a plugin.** New vendor, new analytic, new government database — each is a new implementation of an existing port, not a new fork of the platform.
4. **Degrade, never drop.** Weak link → reduce frame rate → metadata-only → store-and-forward. A camera on a 128 kbps rural link still produces usable intelligence.
5. **Every claim is auditable.** Every alert traces to a frame, a PTS, a camera, a model version and a confidence. Every human action is logged in a hash-chained audit trail.
6. **Time is PTS.** Wall-clock arrival time is never a source of truth anywhere in the system.
7. **Privacy is a feature, not a disclaimer.** Default-blur, purpose-bound access, reason-for-access capture, retention enforcement.

---

2. Architecture Decision Records

ADR-001 — Edge-first analytics, metadata to the centre

Context. 80,000 cameras, sites up to ~1,000 km apart, highly variable bandwidth. **Decision.** Decode and run inference as close to the camera as possible (district node or site box). Only structured events, snapshots and on-demand clips cross the WAN. **Consequences.** Statewide analytics backbone ≈ **140 Mbps** instead of **160 Gbps** (~1,140× reduction; see 04-SCALE-PLAN §2). Adds distributed fleet-management complexity (model rollout, health, store-and-forward) — accepted, and addressed by the edge

agent design in §5. **Rejected.** Central decode of all streams: needs ~160 Gbps sustained plus thousands of decoder cores; not fundable, not deployable on existing infrastructure.

ADR-002 — Event-driven + selective continuous recording

Context. The jury wants Model-4 capabilities; the physics does not want a statewide archive. **Decision.** Every edge node keeps a rolling local buffer (default 60–300 s). On an event or alert, a pre/post-roll clip is sealed, SHA-256 hashed, and pushed centrally as evidence. Departments may additionally flag a small set of **critical cameras** for continuous central recording with tiered hot/warm/cold lifecycle. **Consequences.** Full record/playback/scrub/export UX with ~0.1% of the storage. Non-critical historical footage remains in the department's existing retention (7–15 days) and is retrievable by federated request. **Rejected.** Statewide 24×7 central archive: 1.73 PB/day, 12.1 PB for a 7-day hot tier.

ADR-003 — Adapter/plugin federation

Decision. A single `CameraSource` port with pluggable drivers (ONVIF, RTSP, HLS/WHEP, Milestone, Genetec, Hikvision, Dahua, CSV/offline). Drivers are discovered by entry-point registration and declare capabilities. **Consequences.** Onboarding a 27th department is a driver, not a release. Capability negotiation means the UI can grey out PTZ/playback where unsupported instead of failing. **Rejected.** Per-vendor monoliths, and "everything is RTSP" (loses PTZ, events, and playback APIs that departments already paid for).

ADR-004 — Python services, TypeScript UI, unforked Go binaries

Decision. All first-party services in Python 3.13 (FastAPI/asyncio). Frontend in TypeScript. MediaMTX, NATS, Postgres, MinIO consumed as binaries/images we configure but never modify. **Consequences.** A solo maintainer holds one mental model. Python's throughput ceiling is mitigated by process-per-camera-group workers, hardware decode, ONNX Runtime releasing the GIL during inference, and pushing fan-out to NATS. Where Python genuinely cannot keep up (relay, bus), we are using compiled off-the-shelf components. **Rejected.** Go/Java services for throughput: correct at a large team, wrong for one person with a fixed calendar.

ADR-005 — PTS is the only clock

Decision. Frames are wrapped in a `Frame(pts_ms, stream_epoch, monotonic_seq, …)` value object; analytics code has no access to `time.time()` for measurement. **Consequences.** Trackers, dwell/speed estimates and route timelines stay correct despite the gateway's GOP-replay burst on connect. Mapping PTS → absolute wall time happens once, at the edge, via a per-stream epoch anchor with drift correction, and that anchor is recorded with every event for forensic defensibility.

ADR-006 — One primary datastore

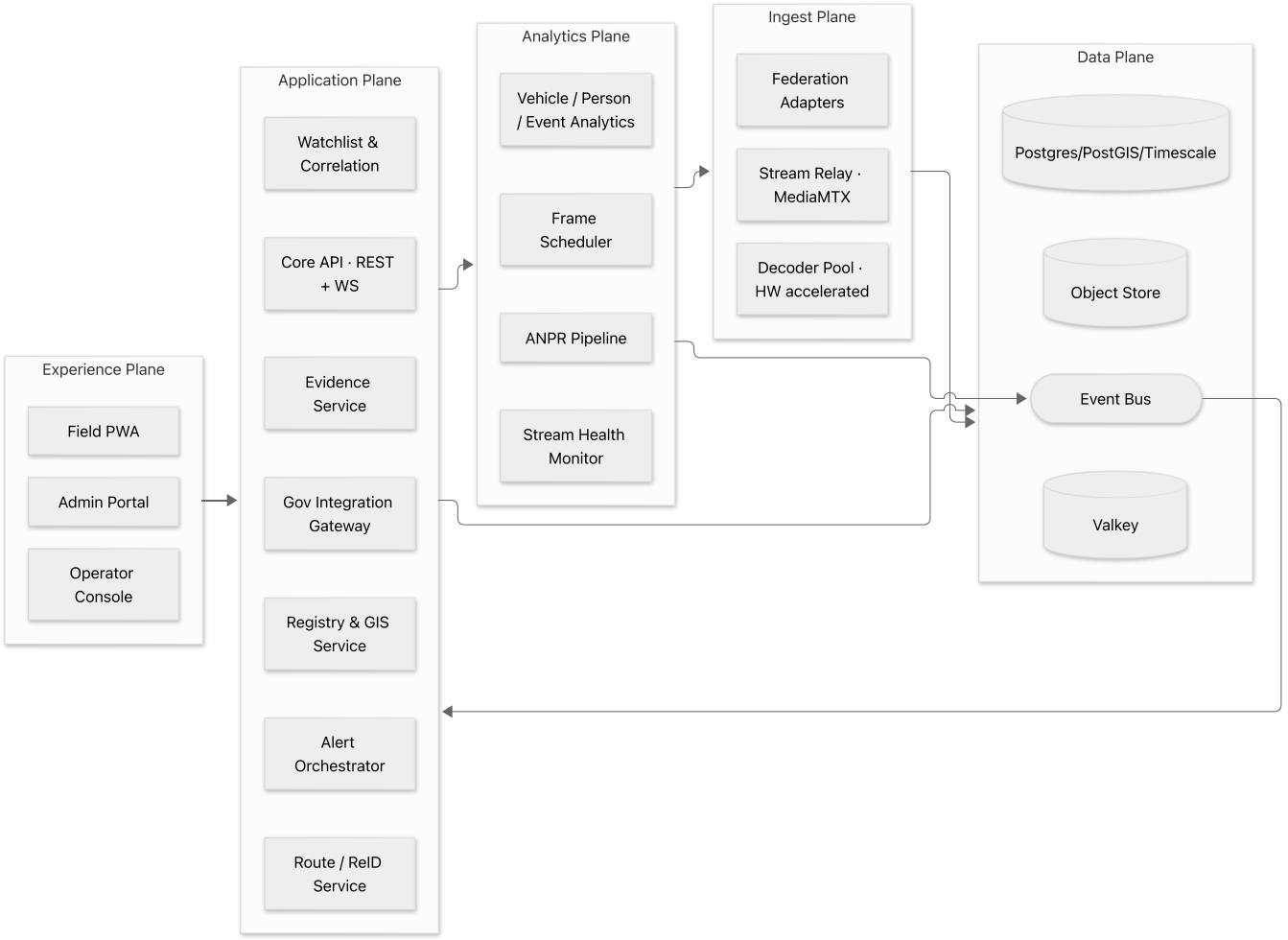
Decision. PostgreSQL 17 + PostGIS (spatial) + TimescaleDB (detection/health hypertables) + `pg_trgm` (fuzzy plate search). OpenSearch is documented in the scale plan, not deployed now. **Consequences.** Simpler ops, transactional consistency between registry/watchlist/events, and sub-second plate search at demo scale. Migration path to OpenSearch is behind a `SearchPort` interface.

ADR-007 — Mobile as an installable PWA

Decision. One React codebase, three shells (Console / Field / Admin). Mobile ships as an installable PWA with Web Push and an offline queue. **Consequences.** No app-store/signing overhead, instant demo on any judge's phone via URL, offline-capable via service worker. Native-only capabilities (background geolocation, native notifications on iOS < 16.4) are the accepted trade.

---

3. Logical planes



Each plane is independently scalable and independently deployable. In the M4 demo all five run on one machine via Compose; in production the Ingest+Analytics planes replicate per district while the Application+Data planes are centralised and HA.

---

4. Ingest Plane

4.1 Federation adapter framework (Model 3)

```
class CameraSource(Protocol):
    driver_id: str                      # "onvif" | "rtsp" | "milestone" | ...
    capabilities: SourceCapabilities     # live, playback, ptz, events, snapshot, motion

    async def discover(self, scope: DiscoveryScope) -> list[CameraDescriptor]: ...
    async def open(self, cam: CameraDescriptor, profile: StreamProfile) -> StreamHandle: ...
    async def snapshot(self, cam: CameraDescriptor) -> bytes: ...
    async def playback(self, cam: CameraDescriptor, window: TimeRange) -> StreamHandle: ...
    async def subscribe_events(self, cam: CameraDescriptor) -> AsyncIterator[VendorEvent]: ...
    async def health(self, cam: CameraDescriptor) -> SourceHealth: ...
```

Drivers shipped:

Driver	Status	Notes
`rtsp`	Full	TCP-forced, the universal fallback; used for the government test grid
`hls` / `whep`	Full	Firewalled networks and browser preview; test-grid fallback when 8554 is blocked
`onvif`	Full	WS-Discovery, Profile S/T media, PTZ, event pull-point. Covers most modern IP cameras across vendors.
`gov-catalogue`	Full	Reads the government test-grid catalogue; maps per-camera codec/resolution/status into descriptors; drives auto-onboarding and reconciliation. See the callout below for the confirmed real endpoint.
`milestone` / `genetec` / `hikvision` / `dahua`	Contract + conformance-tested mock	Real protocol shape (auth flow, stream URI negotiation, event subscription) implemented against a mock server, so a live SDK swap is a credential change plus a few hours. Documented honestly — no licence, no false claims.
`csv-offline`	Full	Metadata-only onboarding for cameras that cannot stream (registry/gap analysis still works)

> **The real test-grid endpoint** (confirmed from the organisers' integrator guide, superseding the generic placeholders used earlier in planning): catalogue at `GET https://cctv.corp8.cloud/cameras.json`; HLS at `https://cctv.corp8.cloud/<id>/index.m3u8` behind a portal password; RTSP/WHEP served directly from a public IP (`103.250.160.189:8554` / `:8889`) with **every connection authenticated by the registered account's email + access password embedded in the URL** (email percent-encoded — `@` becomes `%40`), not anonymous access as earlier drafts assumed. Implemented in `sentinel_core.config.Settings` (`gov_stream_url`, `gov_hls_url`) and `sentinel_core.gov_catalogue`. Credentials live only in a local, gitignored `.env` (see `.env.example`) — never in source. The exact JSON shape of `cameras.json` itself is still unconfirmed; the parser is deliberately tolerant of common key-name variations and `scripts/verify_gov_catalogue.py` is provided to validate/correct it against the live endpoint once real credentials are available.

A **capability matrix** is surfaced in the UI so operators see exactly what each source supports. A **driver conformance test suite** (`pytest` contract tests) runs against every driver, including the mocks — this is the evidence that the plugin SDK is real.

4.2 Stream relay

MediaMTX per edge node:

- Pulls RTSP-over-TCP from source, republishes as **WHEP/WebRTC** (sub-second, for the live wall), **HLS/LL-HLS** (firewall-safe, mobile), and **RTSP** (for local analytics workers, so one pull from the source serves N consumers).
- **One source pull, many consumers** — critical, because the test grid gives each client its own stream copy and we are told to pace load.
- Records event clips and continuous segments for critical cameras (fMP4), with the playback server exposed behind our Evidence Service for authorisation and audit.
- Publishing endpoints and the control API are disabled in config, asserted at boot — compliance with "consume only".

4.3 Decoder pool and frame scheduler

- Hardware decode: **VideoToolbox** on Apple Silicon; **NVDEC/DeepStream** in production.
- Decode directly to the analytics resolution where the decoder supports scaling — never decode 4K to then resize on CPU.
- **Analytics tiers** (per-camera, configurable, driven from the registry):

Tier	Analytic FPS	Use	Cost
A — Continuous	8–15 fps	Traffic junctions, check-posts, high-value	Highest
B — Sampled	1–3 fps	General public-domain cameras	Medium
C — Motion-gated	0 until motion, then burst to tier B	Low-traffic, godowns, offices	Low
D — Registry-only	none	Viewing + inventory only	Zero

> **Demo profile.** Measured *inference* capacity on Apple Silicon (see 02-ANPR-PIPELINE §6) is ≈220 inferences/s on a base M2 and an estimated 350–440/s on the M4 — roughly 120–145 analysed frames/s for the full ANPR chain. The ~50-camera government grid therefore runs as **10 cameras at tier A (5 fps) + 40 at tier B/C (1 fps, motion-gated) ≈ 90 frames/s**, comfortably inside that inference budget, with all 50 onboarded, viewable and health-monitored. Tier rates above are the *production* profile on L4-class GPUs. This is the same tiering logic at both scales — state it that way rather than apologising for it.
>
> A separate, *decode-only* capacity benchmark (`scripts/bench_capacity.py` , M1) measures how many concurrent RTSP connections a machine can decode before individual streams degrade, independent of inference. Its first runs were on a shared, contended sandbox (see `evidence/M1-CAPACITY-FINDINGS.md` for the full account, including a real methodology bug it caught in itself) and are **not** the authoritative number — re-run it on the actual demo M2/M4 with nothing else competing for CPU before quoting a decode ceiling to a jury.

- **Motion gating** uses decoder-side cues where available (macroblock/motion vectors, GOP size deltas) before touching a neural net — cheap, and it is what makes tier C nearly free.
- **Ragged batching**: frames are grouped by resolution class into batches; there is never a fixed-shape batch across all cameras.
- **Backpressure**: a bounded per-camera queue that drops the *oldest* frame (never blocks the decoder), with the drop rate exported as a metric.

4.4 Resilience behaviours

Behaviour	Implementation
Reconnect	Supervisor task per camera, exponential backoff 2 s → 30 s with jitter, per-camera circuit breaker, state machine `connecting → live → degraded → down`
Join-time decode noise	Errors suppressed to `debug` until first IDR; "first-IDR latency" is a health metric
Loop / scene discontinuity	Detector (frame-histogram distance + PTS regression break) → emit `stream.discontinuity`, reset trackers, ReID gallery and background models
Inter-frame gap	Treated as normal; stall breaker keys on elapsed PTS, not on missing frames
Effective FPS	Measured from PTS in a rolling window; declared FPS is metadata only
Idle captures	Reference-counted; closed after TTL; live open-capture gauge on ops dashboard

---

5. Analytics Plane

5.1 ANPR

Full detail in 02-ANPR-PIPELINE. Summary chain:

```
frame → vehicle detect → multi-object track (ID per vehicle)
      → plate detect (within vehicle ROI) → quad rectification
      → OCR → per-frame candidate
      → temporal voting across the track → single high-confidence plate per vehicle
      → format validation (Indian plate grammar) → normalised plate key
      → ANPR event (plate, confidence, bbox, snapshot, PTS, camera, geo)
```

Temporal voting is the single biggest accuracy lever: one vehicle seen over 20 frames yields 20 noisy reads; character-level weighted voting across them beats any single-frame model.

5.2 Secondary analytics (bonus scoring)

Analytic	Purpose	Notes
Vehicle class + colour + coarse make	ReID fallback when plates are unreadable; richer search filters	Lightweight classifier on the vehicle crop
Person detection + attributes	Crowd/intrusion context	Faces blurred by default at the edge
Zone rules: intrusion, loitering, wrong-way, stopped-vehicle	High operational value, low compute	Polygon zones drawn by operators on the camera view
Crowd density estimate	Law-and-order situational awareness	Tier A cameras only
Camera tamper / blur / blackout / scene-change	Feeds the health dashboard; catches sabotage	Cheap classical CV, runs even on tier C
Face detection — **for blurring only**	Privacy protection of bystanders	**No face *recognition* anywhere in the platform, by design.** Faces are located solely so they can be masked at the edge before the frame leaves the node. See §9.

5.3 Event schema (the system's lingua franca)

```
{
  "event_id": "01JB…",             // ULID, sortable
  "type": "anpr.plate_read",
  "camera_id": "GJ-AHM-0421",
  "device_geo": {"lat": 23.0225, "lon": 72.5714},
  "pts_ms": 1843200,                // authoritative stream time
  "stream_epoch": "2026-09-22T09:14:03.221Z",
  "observed_at": "2026-09-22T09:44:46.421Z",  // derived: epoch + pts, NOT arrival
  "payload": {
    "plate_text": "GJ01AB1234",
    "plate_normalised": "GJ01AB1234",
    "plate_confidence": 0.94,
    "char_confidences": [0.99, 0.97, …],
    "vehicle": {"class": "car", "colour": "white", "track_id": "c-0421-8813"},
    "bbox": [812, 440, 96, 34]
  },
  "evidence": {"snapshot_uri": "s3://…/01JB….jpg", "clip_uri": null, "sha256": "…"},
  "pipeline": {"models": {"det": "…@1.2.0", "ocr": "…@2.0.1"}, "node": "edge-ahm-01"},
  "schema_version": "1.0"
}
```

Every event carries its model versions and node identity — required for forensic defensibility and for A/B-ing model upgrades.

---

6. Application Plane

6.1 Watchlist & correlation engine

Watchlist sources: locally curated lists (stolen, wanted, missing, blacklisted, suspect, BOLO) plus, in production, synchronised feeds from VAHAN/eGujCop. Each entry has type, priority, jurisdiction scope, validity window, requesting officer and case reference.

Matching ladder — each rung is tried in order, and each produces a different alert confidence:

Rung	Method	Example
1	Exact normalised match	`GJ01AB1234` == `GJ01AB1234`
2	OCR-ambiguity-class match	`GJO1AB1Z34` → `GJ01AB1234` (O↔0, Z↔2)
3	Bounded edit distance (Levenshtein ≤ 1–2, position-weighted)	last-character errors are more forgivable than the state prefix
4	Partial / wildcard (trigram index)	`GJ01??1234` from a witness statement
5	Attribute match	white hatchback + partial `…1234` in a time-space window
6	Appearance ReID	same vehicle embedding, plate unreadable at this camera

> **Implemented today:** continuous watchlist correlation (alerts) uses rungs 1–2. Find a Vehicle's route reconstruction uses rungs 1–3, stopping at the first rung that finds anything; rung 3 is Levenshtein ≤ 1, only for plate-length queries, and its hops are returned as *probable* (drawn dashed, with the plate actually read) because a one-character difference can be a different vehicle. Rungs 4–6 are design, not code.

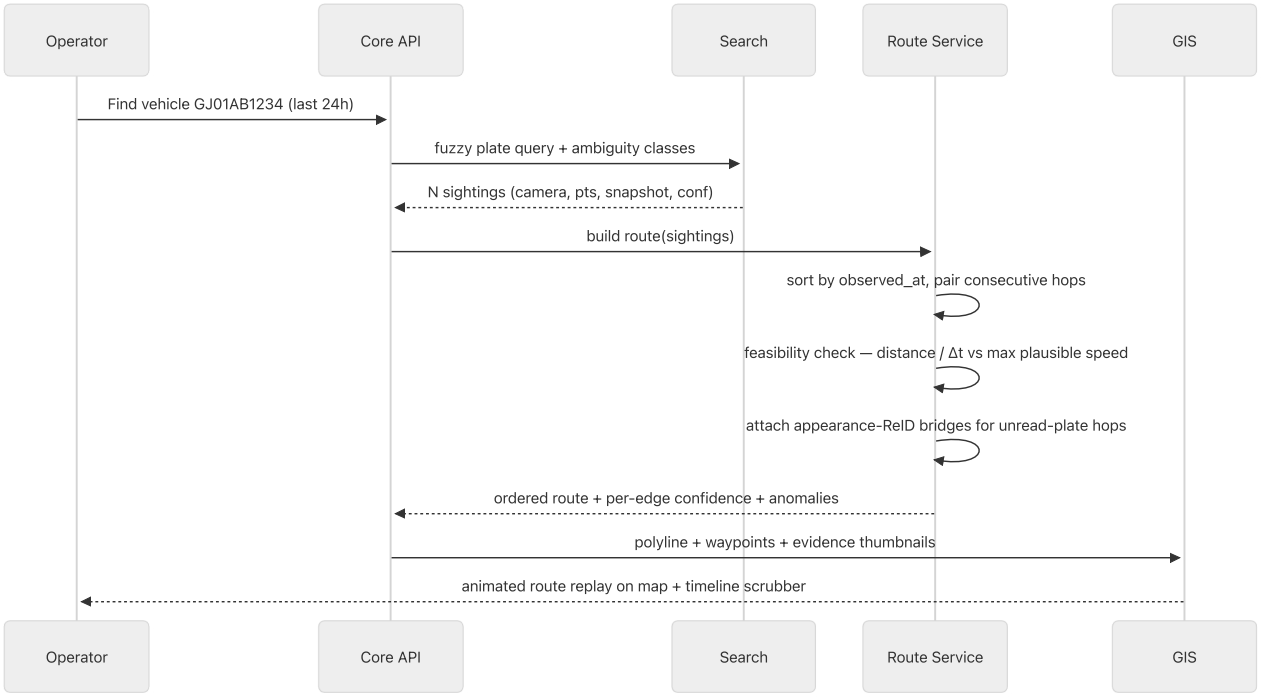
Every rung is scored and the resulting alert shows *why* it matched, in plain language. That explainability is what makes an operator trust it — and it is exactly what a jury probes.

Correlation is continuous, not query-driven: every incoming event is evaluated against the in-memory watchlist index (Valkey-backed, versioned, hot-reloaded on change). Adding a plate to the watchlist triggers both a forward-looking arm **and** an asynchronous retro-scan of historical events — the BOLO moment.

6.2 Alert orchestrator

- **Priority score** = watchlist priority × match confidence × recency × geographic relevance (distance to the requesting jurisdiction) × repeat-sighting boost.
- **Deduplication:** same plate + same camera within a sliding window collapses into one alert with a sighting count, so an operator is never spammed by a car waiting at a red light.
- **Escalation:** unacknowledged high-priority alerts escalate by role and then by channel (console → PWA push → SMS/webhook hook point) on a configurable ladder.
- **Lifecycle:** `new → acknowledged → assigned → in_progress → resolved | false_positive`, each transition audited with actor, time and optional note.
- **False-positive feedback loop:** a dismissal is recorded as labelled training data and shown in a model-quality dashboard — closing the loop is a differentiator and directly answers the "false-match redressal" critique in the press.

6.3 Route reconstruction & cross-camera ReID



- **Feasibility gate:** if two sightings imply 300 km/h, the edge is flagged (cloned plate, OCR error, or clock skew) rather than silently drawn. Road-network distance via OSRM in production; haversine × road-factor at demo scale.
- **ReID bridging:** when a camera reads the vehicle but not the plate, an appearance embedding + spatio-temporal window links it into the route as a *probable* hop, rendered visually distinct from *confirmed* hops.
- **Output:** a shareable, exportable **Movement Report** — map, ordered table of location + timestamp, snapshot per hop, confidence per hop, and a hash manifest. This is literally the eval-day deliverable.

6.4 Government integration gateway

One `ExternalRegistry` port with `lookup_vehicle(plate)`, `lookup_licence(dl_no)`, `lookup_person(...)`, `push_alert(...)`. Implementations: `VahanProvider`, `SarthiProvider`, `EGujCopProvider`, `AfisProvider`, `NafisProvider` (NCRB, national gallery) — each shipped with a **conformant mock** driven by a representative dataset we create, plus a documented request/response contract, retry/circuit-breaker policy, rate limits, field-level audit and PII redaction rules. The message to the jury: *"On the day you grant access, this is a credential change, not a project."*

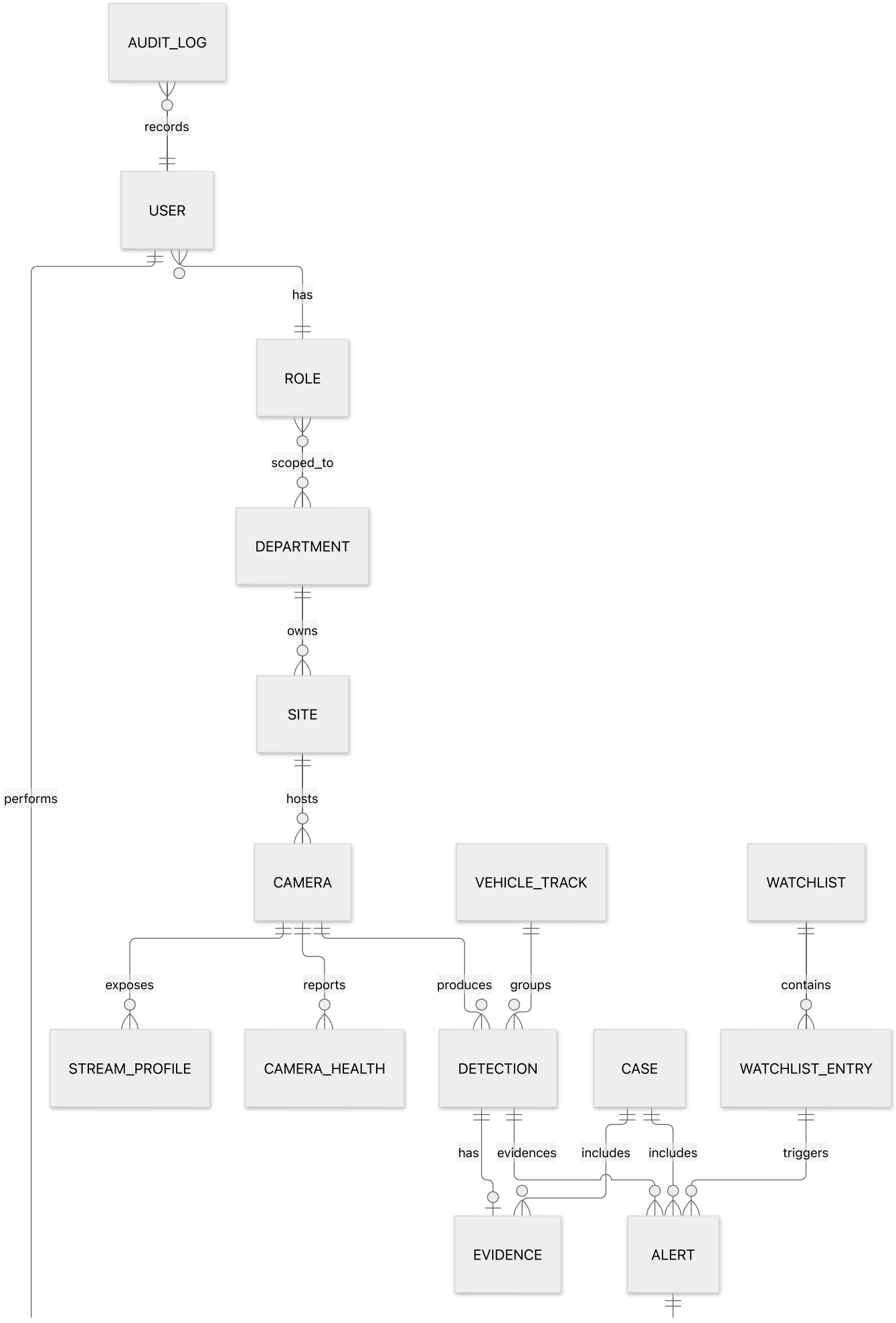
> **Implemented** in `packages/sentinel_core/src/sentinel_core/gov_registry/` (base.py's `ExternalRegistry` Protocol + `resilience.py` 's shared `RateLimiter` / `CircuitBreaker` / `with_retry`, then one file per provider). Each provider talks HTTP to a `base_url` behind an injectable `transport` exactly like the M11 camera drivers do; with no real government credential in this deployment, `transport=None` falls back to a built-in mock transport serving a representative dataset from the same file — so the retry/circuit-breaker/rate-limit code path is genuinely exercised, not bypassed for the demo.
> `VehicleRecord` / `LicenceRecord` / `PersonRecord.audit_safe_dict()` mask PII fields (name, address, chassis/engine number, DOB) for the `RegistryAuditEvent` every call emits. The Admin Portal's Integrations tab (`GET /api/v1/admin/integrations`) drives one real representative lookup per provider live and reports "Connected (mock)" only if that round trip actually succeeded just now.

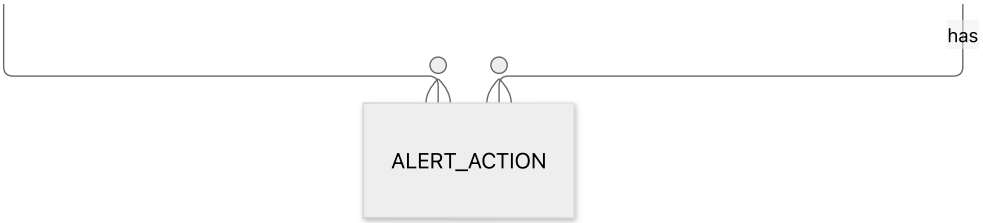
6.5 Evidence service & chain of custody

- Snapshot at detection; on alert, a sealed pre/post-roll clip.
- SHA-256 over the media + a canonical metadata manifest; hashes chained per case so tampering is detectable.
- Every access (view, download, export) is audited with actor, reason and case reference.
- Export bundles: media + manifest + PDF report, suitable for attaching to a case file.

---

7. Data model (core tables)


AUDIT_LOG
records
USER
has
ROLE
scoped_to
DEPARTMENT
owns
SITE
performs
hosts
CAMERA
VEHICLE_TRACK
WATCHLIST
exposes
reports
produces
groups
contains
STREAM_PROFILE
CAMERA_HEALTH
DETECTION
CASE
WATCHLIST_ENTRY
has
evidences
includes
includes
triggers
EVIDENCE
ALERT



Key choices:

- `camera.location` is `geography(Point,4326)`; `camera.fov` is an optional `geography(Polygon)` for true coverage analysis (not just dots on a map) — this is what makes gap analysis credible.
- `detection` and `camera_health` are **TimescaleDB hypertables** partitioned by time, with continuous aggregates powering the dashboards.
- `plate_normalised` carries a `pg_trgm` GIN index plus a generated `plate_ambiguity_key` column (ambiguous characters folded to a canonical class) for rung-2 matching at index speed.
- `audit_log` rows carry `prev_hash` / `row_hash` — an append-only hash chain, verifiable by a CLI command.
- Row-Level Security policies enforce department tenancy in the database itself, not only in application code.

---

8. API surface

`/api/v1`, OpenAPI 3.1 generated from Pydantic models, typed TS client generated for the frontend.

Group	Endpoints (representative)
Registry	`GET/POST /cameras`, `POST /cameras/bulk-import`, `POST /cameras/discover`, `GET /cameras/{id}/health`, `GET /coverage/gaps`, `GET /departments`
Streaming	`GET /cameras/{id}/stream?protocol=whep|hls`, `POST /cameras/{id}/snapshot`, `GET /cameras/{id}/playback?from&to`
Detections	`GET /detections` (filters: plate, camera, time, class, colour, confidence), `GET /detections/{id}`
Vehicle search	`POST /search/vehicle` (exact/fuzzy/partial/attribute), `GET /vehicles/{plate}/route?from&to`, `GET /vehicles/{plate}/report`
Watchlist	CRUD `/watchlists`, `/watchlists/{id}/entries`, `POST /bolo` (arm + retro-scan in one call)
Alerts	`GET /alerts` (stream + filters), `POST /alerts/{id}/acknowledge|assign|resolve|false-positive`, `WS /alerts/stream`
Evidence & cases	`POST /cases`, `POST /cases/{id}/evidence`, `GET /evidence/{id}/download`, `GET /evidence/{id}/verify`
Ops	`GET /health/system`, `GET /metrics`, `GET /nodes`, `GET /compliance/integrator`
Admin	users, roles, departments, API keys, retention policies, `GET /audit`
Integration	`/integrations/vahan/lookup`, `/integrations/egujcop/...` (provider-backed, mock in dev)

Realtime: a single authenticated WebSocket multiplexing `alerts`, `camera_health`, `detections` (filtered), and `node_status`, with per-topic subscribe/unsubscribe and server-side rate limiting.

---

9. Security, privacy & governance

Security

- mTLS between edge nodes and core; short-lived node identity certificates; node enrolment with an approval step.
- OIDC SSO (Keycloak) with RBAC + ABAC: role × department × jurisdiction × purpose. Postgres RLS as the second line.
- TLS 1.3 everywhere; AES-256 at rest for object storage and DB volumes; secrets via environment/secret manager, never in the repo.
- Network segmentation: camera VLANs are reachable only by edge nodes; the core never talks to a camera directly.
- Rate limiting, input validation at the schema boundary, signed URLs with short TTLs for media, CSP/HSTS on the web tier.

- Supply chain: pinned dependencies, SBOM generation, `pip-audit` / `npm audit` in CI, container image scanning.

Privacy (and this is scored — the public critique of such systems is exactly this)

- Faces and, optionally, non-target vehicles are **blurred at the edge by default**; the unblurred frame never leaves the node unless an authorised reveal occurs.
- **Reveal-on-authorisation:** unmasking requires a role, a case reference and a typed reason; high-sensitivity reveals can require four-eyes approval. Every reveal is audited.
- **Purpose limitation:** watchlist entries carry a purpose and an expiry; expired entries stop matching automatically.
- **Retention policy engine:** per-data-class retention (raw clips, snapshots, events, audit) enforced by a scheduled job, with a visible countdown in the admin UI.
- **False-match redressal:** every alert dismissal is recorded; a subject-impact report can be produced per person/vehicle.
- **DPDP Act 2023 alignment** and a documented data-protection impact assessment section in the HLD.

---

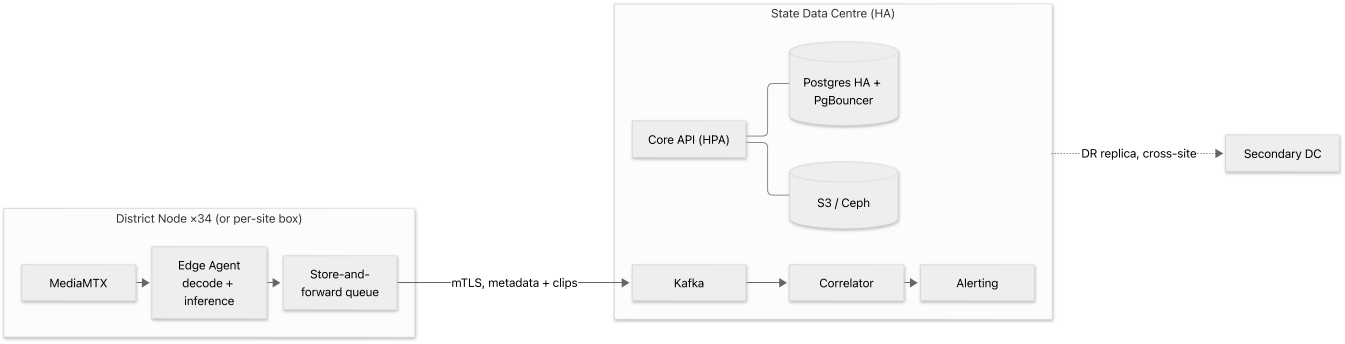
10. Deployment topology

Demo (M4, one machine, `docker compose up`)

```
compose: postgres(+postgis,+timescale) · valkey · nats · minio · mediamtx
         core-api · edge-agent(×N by tier) · correlator · alerting · web
```

Edge agent and core run side by side but communicate **only over the bus and the API** — identical code path to production, which is what makes the demo architecturally honest.

Production (per district edge + central core)



Kubernetes with Helm charts; HPA on analytics workers keyed on queue depth; PodDisruptionBudgets; Postgres HA (Patroni) with streaming replication to the DR site; object storage cross-region replication; RPO 15 min / RTO 1 h targets, justified in the scale plan.

---

11. Repository layout

Directories marked ✅ exist today (M0); the rest arrive with their milestone.

```
sentinel-hackathon/
├─ apps/
│  └─ web/                      # React 19 + Vite · Console / Field / Admin shells   (M6)
│     ├─ src/shells/{console,field,admin}/
│     ├─ src/design-system/     # tokens, primitives, motion
│     └─ src/features/{map,wall,search,alerts,registry,playback}/
├─ services/
│  ├─ core_api/             ✅ # FastAPI · REST + WS · auth · OpenAPI
│  ├─ edge_agent/           ✅ # capture · decode · schedule · infer · publish
│  │  ├─ adapters/             # onvif, rtsp, hls, gov_catalogue, vendor shims     (M1/M11)
│  │  ├─ pipeline/             # clock, scheduler, batching, backpressure          (M1)
│  │  └─ analytics/            # anpr, vehicle, zones, tamper                      (M2/M13)
│  ├─ correlator/              # watchlist matching ladder, retro-scan             (M5)
│  ├─ alerting/                # prioritisation, dedupe, escalation, notify        (M5)
│  ├─ route_service/           # cross-camera ReID + route reconstruction          (M8)
│  └─ evidence/                # clips, hashing, export bundles                    (M7)
├─ packages/
│  ├─ sentinel_core/        ✅ # schemas, PTS clock, plate rules, bus port, config
│  └─ ts-api-client/           # generated from OpenAPI                            (M6)
├─ models/                     # download + ONNX export + benchmarks (no weights in git) (M2)
├─ infra/
│  ├─ compose/              ✅ # docker-compose + mediamtx.yml + db init
│  └─ k8s/                     # Helm charts (production story)                    (M12)
├─ deliverables/               # PPT, HLD PDF, demo scripts, evidence reports      (M14)
├─ docs/                   ✅ # these documents
├─ evidence/               ✅ # benchmark CSVs, accuracy tables, capacity curves
├─ scripts/                ✅ # benchmarks, evidence-report generator, audit verifier
└─ tests/                  ✅ # unit · adapter conformance · integration · compliance
```

`packages/sentinel_core` is deliberately free of OpenCV, torch and FastAPI: it is imported by both the edge plane and the core plane, so keeping it light keeps the edge agent's container small and its cold start fast.

---

12. Testing strategy

Layer	What	Tool
Unit	matching ladder, plate normalisation, PTS clock, priority scoring	pytest
Adapter conformance	every `CameraSource` driver satisfies the same contract, incl. mocks	pytest contract suite
Compliance	asserts TCP forced, no wall-clock in analytics paths, backoff bounds, catalogue-driven discovery, publish disabled	pytest (mirrors the official checklist, one test per line)
Integration	ingest → event → match → alert → route, against a local synthetic RTSP grid	pytest + MediaMTX fixture
Resilience	kill a feed mid-run, force a loop cut, block port 8554, slow the WAN	scripted chaos harness
Accuracy	ANPR precision/recall on a held-out labelled set; tracked per model version	benchmark script → CSV/plot
Load	N cameras × M fps on the M4 until degradation; published capacity curve	benchmark harness
E2E UI	critical journeys: BOLO, wall, onboarding wizard	Playwright

A **local synthetic RTSP grid** (MediaMTX looping sample videos with deliberately mixed codecs/resolutions/frame rates) lets you develop without touching the government gateway and lets you reproduce most quirks — decoder warnings at join, gaps, H.265, 4K vs CIF — on demand. One quirk it does **not** faithfully reproduce: ffmpeg's own loop mechanism renumbers timestamps continuously rather than resetting them at the loop point, so a genuine PTS discontinuity has to be tested at the unit level (`StreamClock` 's own tests construct one directly) rather than end-to-end against this fixture — see `evidence/M2-SYNTHETIC-GRID-DISCONTINUITY-FINDING.md` . Build the fixture on day 2 regardless; it pays for itself many times over on everything else.

Next: 02-ANPR-PIPELINE — model selection and accuracy engineering.

02 — ANPR Pipeline

Sentinel · Model selection, accuracy engineering and Apple Silicon execution

> All latency figures marked **(measured)** were benchmarked on a **base M2 (4P+4E, 16 GB)** with `onnxruntime 1.30.0`. The demo machine is an **M4**, which should land roughly 1.5–2× higher — but that multiplier is an *inference, not a measurement*. **Re-run the benchmark scripts on the M4 and quote only measured numbers to the jury.**

> **Planning-figure correction (M15).** Figures of "~50 cameras" and references to a `GET /api/ingest` catalogue endpoint in this document date from planning, before the grid was accessible. The real grid assigned to us has **30 cameras**, and our host serves its catalogue at **`GET /cameras.json`** behind a session-cookie login — the integrator guide's `/api/ingest` path 404s there, because that guide is a shared template with `<host>` placeholders. The planning text is left as written rather than quietly rewritten, so the record of what was assumed stays visible; where this document states a present-tense fact, trust the README and `evidence/`.

> **Implementation status (M2, this session):** the pipeline described below is built, tested and verified against real models — not just researched. See `services/edge_agent/src/edge_agent/analytics/` (`vehicle_detector.py`, `tracker.py`, `plate_reader.py`, `voting.py`, `pipeline.py`), 35 unit tests covering pre/post-processing and voting with constructed data (no model weights needed for CI), plus three real-model verifications: a static-image smoke test (`scripts/smoke_test_anpr.py`), a 500-frame live-capture integration run against the synthetic grid with zero crashes, and a measured full-chain throughput of **~14.7 fps (67.9 ms/frame)** on this base M2 (one vehicle/frame; CoreML EP for both detectors, CPU EP for OCR, per the execution-provider policy in section 3). **Accuracy on Indian plates is now measured** on a hand-labelled holdout from the live camera grid: 65% exact per crop and 78% after voting, on legible plates (`evidence/ANPR-ACCURACY.md`). That measurement replaced `cct-s-v2` with `cct-xs-v2`, added plate-box padding and Indian-grammar correction; the previous configuration read 8.6%. Tracking is a SORT-style Kalman+IoU tracker (a documented simplification of full ByteTrack — see section 2.1); a keypoint-based rectification stage (section 2, step 4) was deferred as a "Should" enhancement since fast-alpr's built-in crop handling already reads cleanly on frontal test imagery.

---

1. Pipeline

```
RTSP (TCP) — VideoToolbox decode — frame scheduler (PTS-driven, motion-gated)
   │
   ├─[1] Vehicle detection ........ YOLO26n / YOLO11n ONNX @640, ORT CoreML EP
   ├─[2] Multi-object tracking .... SORT-style tracker (Kalman+IoU) → stable vehicle_track_id
   ├─[3] Plate detection .......... yolo-v9-t-640-license-plate-end2end, CoreML EP
   │                                 (run on the vehicle ROI, not the full frame)
   ├─[4] Rectification ............ YOLOv8m-Pose 4-corner keypoints → perspective warp
   ├─[5] OCR ...................... fast-plate-ocr CCT-S, ORT **CPU EP**
   ├─[6] Temporal voting .......... per-slot confidence accumulation over the track
   └─[7] Validation ............... Indian plate grammar + RTO state-code check
   │
   └──→ one high-confidence plate per vehicle → ANPR event → correlation
```

Why two detection stages rather than one full-frame plate detector. Detecting the vehicle first gives a track ID, which is what makes temporal voting possible; it shrinks the plate-detection search region (≈2 ms on an ROI versus 7.7 ms full-frame); and it yields vehicle class/colour attributes that the correlation engine uses for re-identification when the plate is unreadable. The extra stage pays for itself three times over.

Why temporal voting is the single biggest accuracy lever. A vehicle is in frame for 1–3 seconds. At 5 analytic fps that is 5–15 independent reads of the same plate. Per-character confidence voting across those reads beats *any* single-frame model, including much larger ones — and `fast-plate-ocr` returns per-character confidences natively, so it is cheap to implement.

2. Model selection

#	Stage	Choice	Licence	Measured latency (M2)	Notes
1	Vehicle detection	**YOLO26n** ONNX @640 via ORT **CoreML EP**	AGPL-3.0	**8.70 ms / 114.9 fps** (YOLO11n, measured)	YOLO26n COCO mAP50-95 40.9, 2.4 M params. Apache alternative below.
2	Tracking	**SORT-style tracker** (Kalman filter + Hungarian IoU assignment), our own ~150-line MIT implementation	MIT (vendored)	negligible	Single-stage association — the honest simplification of full ByteTrack's two-confidence-stage matching, which is a documented, bounded upgrade (not a hidden gap). BoxMOT (AGPL) was avoided entirely; needs no ReID net. Upgrade path: two-stage association, or BoT-SORT (HOTA 69.68) / OccluBoost (HOTA 71.10) via BoxMOT if the AGPL trade is later accepted.
3	Plate detection	`ankandrew/open-image-models` `yolo-v9-t-640-license-plate-end2end`, CoreML EP	**MIT**	**7.69 ms / 130 fps** full-frame (measured)	Repo-reported P 0.966 / R 0.896 / mAP50 0.958. The only *permissively licensed pretrained* plate detector found.
4	Rectification	`PrasannaBAImodel/license-plate-keypoint-detection` (YOLOv8m-Pose, 4 corners) → `cv2.getPerspectiveTransform`	Apache-2.0	~10 ms, only on plate crops	Pose mAP50 0.9264 / mAP50-95 0.9137. **Highest-ROI single addition for oblique CCTV angles.**
5	OCR	`fast-plate-ocr` `cct-xs-v2`, ORT **CPU EP**	**MIT**	**1.65 ms** (measured)	Purpose-built for plates; first-class Keras-3 fine-tuning CLI that runs on a Colab T4. Chosen over the larger `cct-s-v2` (10.87 ms) because it read 30 of 81 labelled Gujarat plates exactly where `cct-s-v2` read 7 (`evidence/ANPR-ACCURACY.md`).
5b	OCR fallback	**PARSeq** on low-confidence crops	Apache-2.0	~14.87 ms	IIIT5K 99.00 / combined 95.95. Last commit Apr 2024 — usable, unmaintained. Selective second opinion only.
6	Attributes / ReID	**FastReID** (VeRi vehicle ReID)	Apache-2.0	—	Unmaintained since Jul 2024 but the best Apache-licensed option; used for cross-camera bridging.

End-to-end `fast-alpr` measured at 30.00 ms/frame (33 fps) on a 1080p frame with default provider selection. With the execution-provider fix in §3 this should fall to roughly **19 ms/frame (~52 fps)**.

2.1 Framework

Build on `ankandrew/fast-alpr` (MIT, 805★, actively maintained). It exposes pluggable `BaseOCR` / detector interfaces, so our fine-tuned weights and the PARSeq fallback drop in without forking.

Steal Frigate's heuristics wholesale (`blakeblackshear/frigate`, MIT — its LPR is the same architectural shape: YOLOv9 plate detector + OCR): `min_area` ≈ 1000 px so 32×32 noise is discarded · `detection_threshold` 0.7 · `recognition_threshold` 0.9 · a 0–10 `enhancement` knob (contrast/sharpen/denoise) applied before OCR · `min_plate_length` · fuzzy known-plate matching with wildcards · a `debug_save_plates` mode that is invaluable when tuning. These are years of field-tuning available for free.

2.2 Rejected

Rejected	Why
OpenALPR	**Dead** — last commit 30 Jul 2020, Tesseract-era LBP cascades. Its 11.4k stars are historical.
`parkpow/deep-license-plate-recognition`	A client for the *paid* Plate Recognizer API. Not open-source, and would fail the mandate.
RF-DETR for vehicle detection	Apache-2.0 and more accurate (COCO AP 48.4), but **broken on CoreML EP — returns 0 detections** and is slower than CPU (8.7 fps). `open-image-models` already hard-codes its exclusion from CoreML.
TrOCR / VLM-based OCR	62 M params minimum for a stage that runs on every tracked vehicle. 30× over budget.
Neural plate super-resolution (Real-ESRGAN)	Generic SR **hallucinates characters** on 56×34 px crops and produces confident wrong reads. In a police system that is a liability, not a feature. Use rectification + temporal voting + contrast enhancement instead.
NVIDIA DeepStream	Proprietary EULA. Named in the production scale path only, and flagged as non-open-source.
Janus	GPLv3, Linux-first, RTSP only via a plugin. MediaMTX (MIT) is strictly better for our use.

---

3. Execution-provider policy ⚠️

This is the highest-value operational finding of the research, and it is counter-intuitive enough that it belongs in the HLD:

Stage	Provider	Measured effect
Vehicle detection (YOLO ONNX)	**CoreML EP**	32.62 ms → **8.70 ms** (3.7× faster)
Plate detection (YOLOv9 ONNX)	**CoreML EP**	38.99 ms → **7.69 ms** (5.1× faster); `yolo-v9-s-608` : 95.62 → 11.13 ms (**8.6×**)
OCR (CCT models)	**CPU EP — explicitly pinned**	CoreML makes it **2–3× *slower*** (1.65 → 5.46 ms for `cct-xs` ; 10.87 → 21.82 ms for `cct-s`)

The `fast-plate-ocr` documentation claims CoreML gives a 5× speedup — **that claim applies only to the legacy CNN models.** The newer CCT (Compact Convolutional Transformer) models regress on CoreML because attention operators partition badly and ANE dispatch overhead dominates a sub-2 ms model. *Pin the OCR session to `CPUExecutionProvider` explicitly; do not rely on automatic provider selection.*

3.1 Known-broken paths — avoid these before they cost a day

1. **CoreML `.mlpackage` export of YOLO11/YOLO26 fails** on `ultralytics 8.4.158` + `coremltools 9.0` (the C2PSA attention block: *"only 0-dimensional arrays can be converted to Python scalars"*). **Workaround: export to ONNX with `opset=17, simplify=True` and use the ORT CoreML EP** — that path works and is fast.
2. **`yolo-v9-t-384` + CoreML throws on zero-detection frames** (*"dynamic shape {-1} but runtime shape {0} has zero elements"*). `open-image-models` catches it and logs a warning — meaning you **silently lose that frame**. Fix: use the 640 model, and/or set `RequireStaticInputShapes=1` .
3. Official macOS ORT wheels ship the CoreML EP built in — no custom build required.

---

4. India-specific work ⚠️ *this changes the plan*

The finding: the shipped `cct-s-v2-global-model` config declares `plate_regions` covering 66 countries — **and India is not among them.** Out-of-the-box OCR accuracy on Indian plates is therefore *unvalidated*, and no credible pretrained Indian ANPR weights exist anywhere (GitHub search returns only single-digit-star student projects with self-reported, unverifiable accuracy claims and, mostly, no released weights).

Consequence for the plan: your stated preference was "start off-the-shelf, fine-tune only if accuracy is poor". That remains the right *sequence* — but treat fine-tuning as **probable, not contingent**, and budget the 1–2 days for it up front in milestone M2. The global model is a warm start, not a solution.

And regardless of accuracy: you currently have no honest way to state an accuracy number. There is no public Indian plate benchmark with real data behind it. Building your own 500–1,000-crop Gujarat holdout set from the test-grid cameras is therefore not optional — it is the only way to put a defensible number on a slide, and almost no competing entry will have one.

4.1 Configuration deltas

```
# plate_config.yaml — fast-plate-ocr, India
max_plate_slots: 11        # GJ01AB1234 = 10; BH-series 22BH6517A = 9; margin for stacked truck plates
alphabet: '0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ_'
img_height: 64
img_width: 128
image_color_mode: rgb      # keep RGB — plate colour encodes vehicle class in India
```

4.2 Plate grammar and post-processing

```
STANDARD = r'^[A-Z]{2}[0-9]{1,2}[A-Z]{1,3}[0-9]{4}$'   # GJ01AB1234
BH_SERIES = r'^[0-9]{2}BH[0-9]{4}[A-Z]{1,2}$'          # 22BH6517A
```

Plus: strip the `IND` HSRP prefix and the hologram/chip region before OCR · validate the two-letter prefix against the 36 state/UT RTO codes (a prefix that is not a real RTO code is a misread, and knowing that fixes it) · handle two-line plates (common on motorcycles and commercial vehicles) by detecting the line break and concatenating · use **plate colour as a class signal**: white/black = private, yellow/black = commercial, black/yellow = rentable, green/white = EV. That colour signal is free, uniquely Indian, and it is a genuinely differentiating detail to show a jury.

4.3 Confidence and ambiguity

Every read carries per-character confidence. Characters below threshold are marked uncertain rather than guessed, and are surfaced in the UI at reduced opacity (see 03-UX-DESIGN §2.2). The correlation engine then matches on **ambiguity classes** rather than on exact strings:

```
0↔O↔D    1↔I↔L    2↔Z    5↔S    8↔B    6↔G    4↔A
```

The normalised plate key folds these classes, so `GJ01AB1Z34` and `GJ01AB1234` collide on a single index lookup. This is what turns a mediocre OCR read into a successful watchlist hit — and it is the reason the matching ladder in 01-ARCHITECTURE §6.1 is a ladder rather than an equality test.

---

5. Datasets for fine-tuning

Licence-clean and verified to contain data:

Dataset	Size	Licence	Use
`ridham2k4/npds-number-plate-detection` (Kaggle)	16,989 train / 1,535 val / 785 test, 20,597 boxes	**CC0**	⭐ Primary Indian plate-*detection* set. Source-grouped split, no augmentation leakage. Updated Sep 2026.
`paneraghanshyam/gujarat-vehicle-number-plates-yolo-ready` (Kaggle)	45.9 MB	**Apache-2.0**	⭐ **Gujarat-specific** formats, fonts and layouts. Directly on-point for SCRB.
`abtexp/synthetic-indian-license-plates` (Kaggle)	18,000 images, all 36 states/UTs	**CC0**	⭐ OCR pretraining, especially for rare RTO codes
`mabo7237/license-plates-700k` (HF)	99,744 files	**MIT**	Large global OCR pretraining corpus
`detectRecog/CCPD`	>300k	**MIT**	Backbone pretraining only (Chinese plates — different aspect ratio and charset)
`thundarstrom/traffic-vehicle-detection` (HF)	23,319 images, 215,109 boxes, UA-DETRAC	**CC BY 4.0**	⭐ Real fixed-intersection CCTV — exactly our camera geometry

Licence traps — do not touch, this is a government deliverable: `dataclusterlabs/indian-number-plates-dataset` (Kaggle says CC0, the HuggingFace mirror of the same data says **CC BY-NC-ND** — NoDerivatives forbids fine-tuning; conflicting licences on identical data means avoid entirely) · `saisirishan/indian-vehicle-dataset` (CC BY-NC-ND, popular but unusable) · `unidpro/*` (NC-ND commercial teasers) · any dataset with an *Unknown* licence, including `adityamangal98/HR-LR-images-of-the-Indian-License-plates`.

A caution worth knowing: the HuggingFace org `thundarstrom` publishes beautifully specified Indian ANPR datasets (an 18,537-crop OCR corpus, a frozen 3,034-crop "DashCop" benchmark with median 56×34 px crops). **Their file manifests contain only `.gitattributes` and `README.md` — the images were never uploaded.** The cards describe data that does not exist. It is the best-designed Indian ANPR data plan available and would be worth an email to the author, but do not plan around it.

5.1 Colab fine-tuning recipe

```
Detector:  CCPD-pretrain the YOLOv9 backbone
           → fine-tune on ridham2k4/npds + paneraghanshyam/gujarat  (~18k Indian images)
OCR:       pretrain cct-s on abtexp synthetic (18k) + mabo7237 (700k)
           → fine-tune on real Indian crops harvested from the test grid
Hardware:  single Colab T4/A100 is sufficient for both
Output:    ONNX export → validate CoreML EP on the M4 → version and pin in the event schema
```

Harvest training crops from your own pipeline: every detection already saves a snapshot, so a half-day of grid running produces thousands of real Gujarat crops at exactly the camera geometry you will be evaluated on. Label a stratified sample (day/night, angle, distance, motion blur) as the holdout; use the rest for fine-tuning.

---

6. Capacity model for the demo

Measured concurrency ceiling on base M2: a shared ORT session with the plate detector on CoreML EP saturates at **≈220 inferences/second** (1 thread → 128/s, 2 → 210/s, 4 → 222/s, 8 → 221/s). Latency degrades linearly past 2 threads while aggregate throughput stays flat — so the scheduler should use a **small fixed worker pool feeding a shared session**, not a thread per camera.

```
Full chain ≈ 3 inferences per analysed frame (vehicle + plate + OCR)
Base M2 :  220 inf/s ÷ 3 ≈  73 analysed frames/s  →  12–15 streams at 5 fps
M4      : ~350–440 inf/s ÷ 3 ≈ 120–145 frames/s   →  ~20–30 streams at 5 fps  (extrapolated)
```

Fitting the ~50-camera government grid onto one M4 — this is exactly why the tiering in 01-ARCHITECTURE §4.3 exists:

```
10 cameras × 5 fps  (tier A, traffic-facing)     =  50 frames/s
40 cameras × 1 fps  (tier B/C, motion-gated)     =  40 frames/s
                                       Total  =  90 frames/s   ✓ within the ~120–145 M4 budget
```

All 30 cameras are onboarded, monitored, viewable and health-tracked; analytics intensity is allocated by operational value. That is not a compromise to apologise for — **it is the same tiering logic that makes the 80,000-camera plan work**, demonstrated honestly at demo scale. Say so explicitly: *"This is not a limitation of the demo machine, it is the architecture."*

Decode is not the bottleneck — the inference ceiling binds first. Budget one VideoToolbox session per camera at the sub-sampled analytic rate (5 fps), not at the full 25 fps. Note that go2rtc's own testing found M1 **CPU** transcoding beat M1 GPU; measure before assuming hardware paths always win.

---

7. Accuracy engineering checklist

Ordered by return on effort:

1. **Temporal voting across the track** — biggest single win, near-zero cost
2. **Keypoint-quad rectification before OCR** — the highest-ROI *model* addition for oblique CCTV
3. **Format/grammar validation + RTO state-code check** — fixes misreads for free

4. **Frigate's enhancement knob** (contrast/sharpen/denoise) tuned per camera — cheap, effective
5. **Per-camera ROI and minimum plate area** — stop wasting inference on sky and pavement
6. **Reject rather than guess** — mark uncertain characters; a flagged partial read is forensically useful, a confident wrong read is a liability
7. **PARSeq second opinion** on low-confidence crops only — accuracy where it matters, cost where it does not
8. **Per-camera calibration** — a mounting angle is fixed, so plate size and skew are predictable per camera; use it

Evaluation protocol: a frozen holdout set (never trained on), reporting plate-level exact-match accuracy, character-level accuracy, precision/recall for detection, and a stratified breakdown by day/night, angle and distance. Every result is tagged with model versions (which the event schema already carries) so regressions are attributable. Track this in `evidence/anpr-accuracy.csv` from milestone M2 onward — the trend line itself is a persuasive artefact.

---

8. Licensing position

Permissive — safe for Phase-2 commercialisation	Copyleft — fine for the hackathon, blocks closed-source productisation
`fast-alpr` MIT · `open-image-models` MIT · `fast-plate-ocr` MIT · Frigate MIT · MediaMTX MIT · go2rtc MIT · PARSeq Apache-2.0 · PaddleOCR Apache-2.0 · FastReID Apache-2.0 · plate-keypoint model Apache-2.0 · RF-DETR Apache-2.0	**Ultralytics YOLO26/YOLO11 AGPL-3.0** · **BoxMOT AGPL-3.0**

Recommendation: ship the AGPL components (Ultralytics for vehicle detection, optionally BoxMOT for tracking) and **state it plainly in the submission** — the challenge mandates "open-source technologies", and AGPL-3.0 is OSI-approved. Then show the **Apache-only migration path** on a Phase-2 slide: RF-DETR (Apache-2.0) for vehicle detection plus a vendored MIT ByteTrack. Naming the licence consequence *before the jury asks* is exactly the kind of judgement that separates a procurement-ready submission from a hackathon project.

Practical hedge: vendor a standalone MIT ByteTrack from day one (it is ~200 lines and needs no ReID network). That removes the BoxMOT AGPL dependency immediately at no cost, leaving Ultralytics as the only copyleft component — and therefore a single, cleanly swappable one.

---

9. Open gaps — carry these into M2

- **All latency numbers are M2 measurements.** Re-benchmark on the M4 before quoting anything.
- **No verified accuracy for any model on Indian plates exists publicly.** Every figure found (95% detection, 80–99% OCR) is a self-reported claim in a low-star repo with no published evaluation protocol. Build your own holdout set.
- `fast-plate-ocr` **CCT-v2 absolute accuracy is unpublished** — the model zoo gives latency only; the 93.3–94.19% figures belong to the *legacy* models. Do not quote a v2 accuracy number from the repo.
- `.mlpackage` **CoreML export is untested** because it is currently broken; a working export with `MLComputeUnits=CPUAndNeuralEngine` might beat the 8.70 ms ONNX path.
- **Roboflow Universe was unreachable during research** (Cloudflare 403) — check `universe.roboflow.com` manually for additional Indian plate datasets, verifying licences before use.
- **Apple hardware decode-engine counts are unverified** — Apple publishes no authoritative figure. Measure; do not quote.

Back to 00-SOLUTION-PLAN · Next: 03-UX-DESIGN.

03 — Experience Design

Sentinel · Design language, surfaces and screen specifications

> **The bar:** a police constable who has never used a VMS should find a vehicle in under 30 seconds, without training, without reading a manual, and without being afraid of breaking something.

Implemented workspace

The current UI uses a **balanced operations workspace**: monitoring and vehicle investigation have equal prominence. A 94px navy navigation rail keeps every destination labelled while giving operational content most of the viewport. Operators can expand it to 224px; that preference persists on the device. Cobalt identifies active navigation and primary actions. The compact toolbar provides current-route context, local date/time, persistent plate search, a visible command-palette launcher, and preferences. Below 768px, navigation becomes a keyboard-accessible drawer and the toolbar wraps into two rows.

The overview leads with four linked summaries derived from API responses: registered cameras, live cameras, down/degraded cameras, and new alerts returned by the queue endpoint. The queue count is not an all-time or fleet-wide alert total. Unknown and connecting cameras are not counted as live. Errors replace summary values with "Not available"; an empty successful response shows zero.

A plate-entry form launches investigation directly. The network workbench switches between a map and searchable camera list; camera-health buttons filter both views. The activity panel switches between watchlist matches and the latest 12 recorded detections. Clicking a match opens that alert in triage; clicking a detection opens its plate investigation. Records without coordinates remain available in the list and are explicitly excluded from the located-camera count. Camera and detection data refresh every 8 seconds, alerts every 6 seconds; manual refresh retries all three. View tabs support arrow keys, Home and End.

Vehicle investigation keeps a persistent registration-plate query bar above its results. The workbench separates movement map, selectable sighting timeline and evidence inspector, with first/last sightings, mapped-sighting counts, replay and export together. Missing coordinates and missing snapshots are explained rather than silently omitted. Empty searches retain the BOLO action.

Alerts use a master-detail triage workbench: lifecycle filters, plate/camera search, severity filtering and sorting act on the loaded queue. The selected alert exposes matching facts, evidence and lifecycle actions. Refresh and action failures retain context and support retry.

The surveillance workbench has a camera selector, filtering, selectable grid density and single-camera focus. Camera inventory combines health filters, searchable records, and a map with a persistent camera list. Bulk CSV import and catalogue discovery are direct onboarding entry points; the detail workspace retains preview, health and zone creation.

Administration is a section hub with a searchable access register, contextual user creation, watchlists, integration checks, audit verification, and retention impact preview. Changing retention periods invalidates a previous deletion confirmation. Health groups integrator checks and can filter checks needing attention; these checks describe configuration, not camera availability.

The field experience uses a task-first mobile shell with lookup, alerts, reporting and nearby cameras. Lookup results hand off to reporting; reports group vehicle details and optional evidence and still enter the existing offline outbox. Alert cards expose actions on demand; nearby cameras can be searched in map/list views, with distances only when location is available.

Light and dark themes, and English, Hindi and Gujarati, are device-persistent preferences. New sessions start in light mode; an existing dark preference is preserved. The sign-in surface pairs a navy product introduction with a focused credential form, stacking on phones; a labelled password-visibility toggle preserves the entered value. No demo credentials or fabricated readiness claims are displayed. API refresh failures show a retry action and warn that retained data may be stale; missing data is not presented as a healthy zero. The toolbar searches **registration plates**, not arbitrary camera names or places; camera filtering and the command palette are separate controls.

The sections below also describe longer-term product aspirations. Features such as reversible destructive actions, alert sounds, saved wall layouts and cases are not implied to be implemented by this redesign.

Validation: from `apps/web`, run `npm run test:ui` for mocked-browser acceptance tests (first install Chromium with `npx playwright install chromium`). The runner starts an isolated Vite server on port 5188. Run `npm run build` and `npm run lint` for production

and static checks.

The full redesign is covered by 62 browser tests across the original workspace suite and new command, investigation, monitoring, management and translation suites. A 61-test integrated run passed, followed by all 12 investigation tests after the final deep-link regression was added. On shared or resource-constrained machines, use `npm run test:ui -- --workers=1` : concurrent WebGL/browser contexts can contend with live inference. These tests validate UI/API contracts using fixtures, not live OCR accuracy or successful government-stream playback.

1. Design philosophy

Apple's three HIG pillars — **Clarity, Deference, Depth** — translated into a command-and-control context:

Pillar	In a control room	Concretely
Clarity	One glance, one truth	Plates in mono type at 28 px. Severity never encoded in colour alone. No screen shows two competing "most important things".
Deference	The interface supports the video and the map	Navy navigation, pale working surfaces and cobalt actions; semantic colours belong to status. No decorative gradients or logo watermarks over video.
Depth	Layers communicate hierarchy, not decoration	Bordered working surfaces; shadows reserved for dialogs. Dialogs are dismissible and scroll within the viewport.

Four additional principles specific to this product:

4. **Calm under pressure.** The UI must not add adrenaline. No flashing, no red-on-red, no alarm fatigue. A critical alert earns exactly one soft chime, one pulse, and a persistent but quiet presence.
5. **Plain language, always.** `Camera not responding — last seen 4 minutes ago. Retrying automatically.` Never `RTSP DESCRIBE failed: 503`. The technical detail lives behind a "Details" disclosure for the engineer who wants it.
6. **Evidence is always one click away.** Every number, alert and route hop is traceable to a snapshot and a timestamp. Nothing is asserted without proof attached.
7. **Forgiving by default.** Undo over confirm. Destructive actions are reversible for 10 seconds via a toast. Nothing a nervous first-time user does should be unrecoverable.

---

2. Design tokens

2.1 Colour

Light is the default for new sessions. Dark remains a first-class peer for control rooms, and saved preferences always win. Navigation stays navy in both themes.

Token	Dark	Light
Base	`#101725`	`#f3f5fa`
Raised	`#182235`	`#ffffff`
Overlay	`#25334b`	`#e9edf6`
Inset	`#131c2c`	`#f7f8fc`
Primary text	`#edf3f7`	`#17243b`
Secondary text	`#c4cfe0`	`#465772`
Tertiary text	`#b2bed2`	`#556580`
Accent	`#a4bfff`	`#2456d6`
On accent	`#10224c`	`#ffffff`

The complete semantic palette is defined in `index.css`. Text and primary-action contrast are covered by browser tests. Severity is conveyed by icon, label and colour, never colour alone. Blue is reserved for actions, selection and focus; success, warning and failure keep their semantic colours.

2.2 Typography

```
--font-ui:    "Public Sans", system-ui            /* UI, body */
--font-data:  ui-monospace, "SF Mono", monospace  /* plates, timestamps, IDs, coordinates */

--text-xs: 11/16   --text-sm: 13/18   --text-base: 15/22
--text-lg: 17/24   --text-xl: 20/28   --text-2xl: 24/32   --text-3xl: 32/40
```

- All numerals use `font-variant-numeric: tabular-nums` so timestamps and counters never shift width.
- Public Sans regular, semibold and bold are self-hosted under `apps/web/public/fonts`, with their SIL Open Font License. Fonts are included in the PWA precache; no runtime font-service request is required. Hindi and Gujarati use the device's script-capable fallback.
- **Plates are rendered in mono, letter-spaced, with a slashed zero** — a deliberate cue that this is machine-read data, and it kills the 0/O ambiguity visually. Uncertain characters render at reduced opacity with a subtle underline, so an operator instantly sees *which* character the model was unsure about.

2.3 Space, shape, elevation, motion

```
--space: 4px base · 4 8 12 16 20 24 32 40 48 64
--radius-sm: 4px · --radius-md: 8px · --radius-lg: 12px · --radius-xl: 16px · --radius-full
--shadow-1: 0 1px 2px rgb(0 0 0 / .32)
--shadow-2: 0 4px 12px rgb(0 0 0 / .36)
--shadow-3: 0 12px 32px rgb(0 0 0 / .44)
--blur-material: saturate(180%) blur(20px)   /* floating panels over video/map */

--ease-out:    cubic-bezier(0.22, 1, 0.36, 1)
--ease-inout:  cubic-bezier(0.65, 0, 0.35, 1)
--spring-soft: spring(1, 120, 16, 0)   /* panels, sheets */
--spring-snap: spring(1, 300, 24, 0)   /* toggles, chips */
--dur-fast: 120ms · --dur-base: 200ms · --dur-slow: 320ms · --dur-map: 700ms
```

Motion rules: elements enter from where they belong (a detail panel slides from the edge it is anchored to, never fades in from nowhere); nothing animates longer than 320 ms except map camera flights; **`prefers-reduced-motion` disables all transforms and keeps opacity-only transitions.** Layout must never shift after load — skeletons occupy the exact final geometry.

2.4 Sound

Three sounds only, all short, soft and pitched to be audible over room noise without startling: `alert-critical` (two-tone), `alert-normal` (single tone), `success` (soft click). Muted state is one click away and remembered per user. Sound is never the only channel for information.

---

3. Surfaces

Surface	Primary user	Device	Core job
Operator Console	Control-room operator, investigating officer	Desktop 1440–2560 px, often dual monitor	Watch, search, triage, trace
Field PWA	Officer on the ground, checkpost staff	Phone, installable, offline-tolerant	Receive alerts, look up a plate, report a sighting
Admin Portal	Department nodal officer, SCRB administrator	Desktop/laptop	Onboard cameras, manage users/watchlists, audit, policies

One React codebase, one design system, three shells. The Field PWA is not a "responsive squeeze" of the console — it is a separate, drastically reduced information architecture built on the same primitives.

---

4. Operator Console

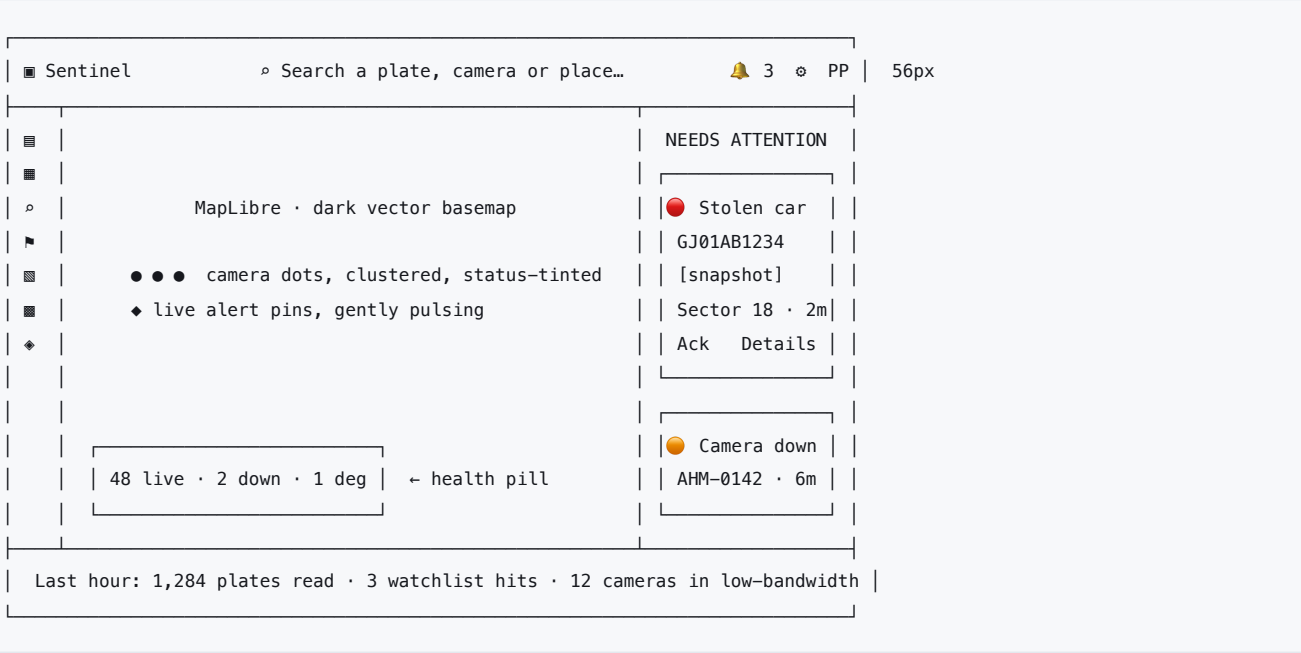
4.1 Information architecture

```
Home (Situational Awareness)  ·  Live Wall  ·  Find a Vehicle  ·  Alerts  ·  Cameras  ·  Cases  ·  Health
```

The implemented destinations are Overview, Live Wall, Find a Vehicle, Alerts, Cameras and Health, with role-gated Admin and a link to the Field PWA. Cases remains planned. A readable sidebar becomes a navigation drawer on phones; the global ⌘K command palette remains available.

4.2 Home — balanced monitoring and investigation

The implemented overview is an interactive network/activity workbench, not a passive map dashboard: view switching, camera search, health filters, plate-entry and linked detection/alert records are described above. The map-first wireframe and decisions below are the earlier concept, retained as background rather than a specification of the current layout.



Decisions that carry the whole product:

- **The map is the application.** Everything else floats above it. This matches how police actually think — geographically.
- **The right rail answers exactly one question: "what needs me?"** It is not an event firehose. Events live in Alerts; the rail shows only what is unacknowledged and relevant to this user's jurisdiction.
- **The search field is the single most prominent control** and accepts a plate, a camera name, or a place. Typing a plate-shaped string immediately offers *"Find vehicle GJ01AB1234"* as the first action — this is the eval-day path, and it is one keystroke deep.
- **Camera dots are tinted by health, sized by importance, clustered by zoom.** Hovering peeks a live thumbnail; clicking opens a compact card with a live preview and three actions (Watch, Recent detections, Details).
- **The bottom strip is ambient, not interactive** — a calm heartbeat that tells an operator the system is alive.

4.3 Live Wall

Implemented workbench: select the cameras to view, filter the selection list, change grid density, or focus one selected camera. Camera details are reachable with labelled buttons as well as the existing double-click interaction. Only visible mounted tiles resolve their streams; empty selection and unavailable streams have explicit states. On phones, the entire wall remains vertically scrollable. Camera registration, zone editing, CSV preview-before-commit and catalogue discovery remain real API actions.

The following capabilities remain the earlier target design; saved layouts, drag reordering, transport switching and playback scrubbers are not implied by the workbench redesign.

- Adaptive grid: 1 / 2×2 / 3×3 / 4×4 / custom, drag to rearrange, layouts saved per user and shareable to a colleague as a link.

- **WebRTC (WHEP) by default; automatic HLS fallback** with a small, honest badge showing the transport and the current latency. A firewall problem should be visible, not mysterious.
- Each tile: name, department chip, live/degraded/offline dot, measured fps, and a detection overlay toggle (plate boxes with the read text pinned beside the vehicle).
- **Only visible tiles stream.** Scrolled-off tiles pause and show the last keyframe — this is both a performance requirement and an explicit organiser instruction ("open only cameras you process").
- Double-click a tile → immersive single view with playback scrubber (for recorded cameras) and recent detections.
- Spotlight behaviour: when a critical alert fires, the wall offers a non-intrusive *"Jump to camera"* chip. It never hijacks the view — hijacking is how operators learn to distrust software.

4.4 Find a Vehicle — the flagship screen

Two states. Nothing else.

State 1 — Ask. A single large input, centred, with a mono placeholder `GJ 01 AB 1234`. Below it, three quiet helpers: *Don't know the full number?* (switches to a per-character grid where any position can be left blank) · *Last 24 hours* ▾ · *All departments* ▾. No advanced-search panel. The advanced options reveal themselves only when the simple path is insufficient.

State 2 — Answer.

```
┌──────────────────────────────────────────────────────────────────────┐
│  GJ01AB1234   ● Stolen vehicle · reported 18 Sep        [Add to watchlist ✓] │
│  7 sightings across 5 cameras · 14:02 → 15:47 · 23.4 km travelled            │
├──────────────────────────────────┬───────────────────────────────────┤
│                                  │  TIMELINE                         │
│    map with route polyline,      │  ● 14:02  Sarkhej Circle     ▓ 0.96 │
│    numbered hops, direction arrows,  ● 14:19  SG Highway Gate 2   ▓ 0.94 │
│    dashed segment = probable     │  ◌ 14:41  Bopal Junction     ▒ 0.71 │
│    (matched by appearance, not plate) │     probable · matched by appearance │
│    ▶ Replay route                │  ● 15:12  Iskcon Crossroad   ▓ 0.98 │
│                                  │  ● 15:47  Prahladnagar       ▓ 0.93 │
├──────────────────────────────────┴───────────────────────────────────┤
│  [thumb] [thumb] [thumb] [thumb] [thumb]   ← evidence strip, click to enlarge │
│                      Export report  ·  Add to case  ·  Share          │
└──────────────────────────────────────────────────────────────────────┘
```

- **Confidence is shown, not hidden.** A filled bar and a number, with a plain-language tooltip: *"Read clearly at this camera"* / *"Two characters uncertain — matched using vehicle colour and timing."*
- **Probable hops are visually distinct** (hollow marker, dashed route segment) and can be toggled off. A jury will ask "how do you know?" — the interface answers before they ask.
- **Replay route** animates the vehicle along the polyline with the timeline scrubbing in sync. It reads as a story, and it is the single most demo-able interaction in the product.
- **Export report** produces the PDF/CSV movement report — the literal eval-day artefact — in one click.
- Empty result is never a dead end: *"No sightings for GJ01AB1234 in the last 24 hours. Watch for this vehicle →"* arms a BOLO, turning a failed search into a standing instruction.

4.5 Alerts

- Inbox metaphor, grouped by priority then recency, with filter chips (Mine · Unacknowledged · Critical · My district · Today).
- **Alert card = Apple notification, evolved:** snapshot thumbnail, plate in mono, *why it matched in one plain sentence* ("Exact match on stolen-vehicle list, added by PI Shah, 18 Sep"), camera and time, and two actions — **Acknowledge** and **Open**.
- Opening an alert slides in a detail panel: the clip with pre/post-roll, the full detection, the vehicle's other sightings, the watchlist entry with its case reference, and the action bar (Assign · Resolve · Not this vehicle).
- **"Not this vehicle" is given equal visual weight to "Resolve"** — making false-positive reporting effortless is how the model improves and how the system stays trusted. It also directly answers the redressal critique levelled at such systems.
- Repeat sightings collapse into one card with a count and a mini-timeline. Never five cards for one car at one traffic light.

4.6 Cameras (registry + onboarding)

- Table ⇄ map toggle, sharing one filter state. Columns: name, department, type, status, last seen, analytics tier, resolution/codec.
- **Onboarding wizard, three steps, non-technical language:**
  1. *Where is it?* — map pin with address autocomplete, plus a draggable field-of-view cone.
  2. *How do we connect?* — big choices first ("Discover on my network" / "I have a link" / "Import a file" / "Connect a department system"). The RTSP URL field is there, but it is not the first thing a nervous user sees.
  3. *What should it watch for?* — plain-language analytics choices ("Read number plates", "Alert on people entering a restricted zone") mapping to the tier system underneath. A **live preview appears the moment credentials validate** — instant feedback that it worked, which is the difference between confidence and a support ticket.
- Bulk import: drag a CSV, get a column-mapping screen with a live preview of the first five rows, per-row validation with inline fixes, and a dry-run before commit.
- **Coverage gap analysis** renders as a heat layer with plain-language findings: *"Sector 21 has no camera coverage within 800 m of the main road"* and an exportable report.

4.7 Health & Ops

- Fleet status board: live/degraded/offline counts, uptime sparklines, first-IDR latency, measured-vs-declared fps, reconnect counts, discontinuity events, queue depth, inference latency, GPU/CPU per node.
- **Integrator Compliance panel** — a live green-tick board mirroring the organisers' pre-submission checklist (TCP forced ✓, PTS-driven timing ✓, backoff active ✓, catalogue-driven discovery ✓, mixed-codec handling ✓, publish disabled ✓). Cheap to build, and it tells a technical jury in three seconds that you read their guide.

---

5. Field PWA

Radically reduced. Four things, large touch targets, one-handed, works on a weak network.

1. **Alerts** — a single scrollable list of cards, newest first, with a full-bleed snapshot. Tap to expand: plate, location with distance-from-me, "Navigate" (opens maps), "Acknowledge", "I see it".
2. **Look up** — a numeric-friendly plate keypad; result shows the vehicle's status (clear / stolen / wanted) as a full-screen colour-and-icon verdict readable at arm's length in sunlight, with the last sighting below.
3. **Report a sighting** — camera capture + auto plate read + GPS, queued offline and synced when connectivity returns.
4. **Nearby cameras** — map of cameras around me, tap to view (HLS, low-bandwidth profile).

PWA specifics: installable with a proper icon set, Web Push for alerts, service-worker offline shell, IndexedDB outbox for queued reports, a low-bandwidth mode that suppresses snapshots and shows text-only alerts, and an explicit "last synced" line so the officer always knows whether they are seeing fresh data.

---

6. Admin Portal

Quieter visual tone (light theme default), denser tables, built for correctness over speed.

- **Departments & sites** — hierarchy, contacts, camera counts, integration status per department.
- **Users & roles** — role matrix editor with a live "what this person can see" preview; jurisdiction scoping on a map.
- **Watchlists** — list management, bulk import, each entry carrying type, priority, purpose, validity window, requesting officer and case reference. Expired entries visibly fade and stop matching automatically.
- **Retention & privacy policies** — per-data-class retention sliders with a plain-language consequence preview ("Event clips will be deleted after 30 days. About 1.2 TB affected.") and default-blur settings.
- **Audit log** — filterable, exportable, with a **"Verify integrity"** button that recomputes the hash chain and shows a green seal. Demonstrating a tamper-evident audit trail live is a quiet showstopper.
- **Integrations** — VAHAN / SARTHI / eGujCop / AFIS / NAFIS connector cards with status, last sync, request volume and a test-connection button.

---

7. Cross-cutting interaction rules

Rule	Why
Optimistic UI with undo, not confirmation dialogs	Acknowledging an alert should feel instant; a 10-second undo toast is safer *and* faster than a modal
Skeletons that match final geometry	Zero layout shift; the screen never "jumps" while an operator is reading
Relative time + absolute on hover, PTS-accurate	"2 min ago" for scanning, `22 Sep 2026, 14:02:18.340 IST` for evidence
Errors state the cause and the next action	"Camera not responding — retrying automatically (next attempt in 8s). Details ▾"
⌘K command palette	Power users fly; everything in it is also reachable by mouse
Keyboard map: `/` search · `A` acknowledge · `←/→` timeline · `Space` play/pause · `Esc` dismiss	Control rooms are keyboard places
ARIA live regions for new alerts, visible focus rings, full keyboard reachability, WCAG 2.2 AA	Accessibility is a government procurement requirement, not a nicety
No dead ends — every empty state teaches and offers the next action	First-run experience decides whether a non-technical user stays
Density toggle (comfortable / compact)	A 4K control-room wall and a 13" laptop are different rooms

Performance budgets (they are part of the feeling of quality, not separate from it): first contentful paint < 1.2 s, interaction latency < 100 ms, map sustained at 60 fps with 5,000 camera markers (deck.gl layer, not DOM), live wall of 16 WebRTC tiles without dropped frames, alert appears in the UI < 2 s after the detection PTS.

---

8. Build approach for a backend-heavy solo developer

This is the honest part of the plan: polish is achieved by *choosing well*, not by hand-crafting CSS.

1. **Steal the hard parts.** Radix/shadcn primitives give correct focus management, keyboard behaviour and ARIA for free. Never hand-roll a dropdown or a dialog.
2. **Tokens first, components second, screens third.** One afternoon spent on `tokens.css` makes every subsequent screen look intentional, because coherence — not ornament — is what reads as "Apple-level".
3. **Constrain the palette brutally.** One accent, four severity colours, a neutral ramp. Most amateur UI looks amateur because it uses too many colours and too many type sizes.
4. **Build a component gallery page early** (`/dev/gallery`) showing every component in every state. It is the fastest way to spot inconsistency, and it doubles as a portfolio artefact.
5. **Motion last, and sparingly.** Add spring transitions only to panels, sheets and the route replay. Animating everything is the signature of a first draft.
6. **Two screens carry the demo** — Home and Find a Vehicle. Spend disproportionate time there; make the rest merely clean and consistent.
7. **Test with a non-technical person.** Hand them a plate number and say nothing. Watch where they hesitate. That hesitation is the design brief.

Next: 04-SCALE-PLAN — the 80,000-camera argument with numbers.

04 — Scale Plan to ~80,000 Cameras

Sentinel · Bandwidth, compute, storage, resilience and cost

> The organisers ask for a credible path to ~80,000 cameras. This document gives the arithmetic, states every assumption, and shows why full centralisation is rejected on evidence rather than on preference. **Every formula is written out so the jury can re-derive the numbers with their own inputs.**

---

1. Baseline assumptions

Parameter	Value used	Basis
Total cameras (target)	80,000	Stated by the organisers
Departments	26	Stated
Districts	34	Gujarat administrative divisions
Typical stream	1080p, 15 fps, H.264 CBR **2 Mbps** (H.265 ≈ 1 Mbps)	Standard government CCTV profile
Existing retention	7–15 days, department-local	Stated in the problem background
ANPR event record	~1 KB JSON (compressed ~300 B at rest)	Schema in 01-ARCHITECTURE §5.3
Evidence snapshot	~30 KB JPEG (plate crop + context)	Measured on the test grid
Alert clip	30 s pre/post-roll, H.265 ≈ **6 MB**	1.6 Mbps × 30 s

1.1 Camera tiering — the key modelling decision

Not every camera needs continuous AI. A PDS godown camera and a highway check-post camera have nothing in common operationally. Tiering is what makes 80,000 tractable.

Tier	Share	Count	Analytic rate	Typical deployment
A — Continuous ANPR	12%	9,600	10 fps	Traffic junctions, check-posts, highways, entry/exit points
B — Sampled	28%	22,400	2 fps	General public-domain, markets, civic areas
C — Motion-gated	35%	28,000	~0.3 fps effective	Godowns, PDS shops, offices, low-traffic premises
D — View + registry only	25%	20,000	0	Indoor/administrative, private consented feeds, view-on-demand

Tiering is an operational policy set per camera in the registry by the owning department, not a hard-coded constant — and it can be changed live (a camera can be promoted to tier A during a VIP movement or a bandobast).

---

2. Bandwidth

2.1 What full centralisation would cost (the rejected option)

```
Aggregate ingest = 80,000 cameras × 2 Mbps = 160,000 Mbps = 160 Gbps sustained
                 = 80 Gbps if every camera were re-encoded to H.265 (they are not; mixed fleet)
```

160 Gbps sustained, 24×7, arriving from sites up to 1,000 km apart, over links that in many talukas are a few Mbps. This is not a budget problem; it is a physics problem. **Rejected.**

2.2 What Sentinel actually sends

Event rate, using the tiering above and conservative traffic assumptions:

```
Tier A:  9,600 cameras × 150 events/hour = 1,440,000 /hr
Tier B: 22,400 cameras ×  20 events/hour =   448,000 /hr
Tier C: 28,000 cameras ×   5 events/hour =   140,000 /hr
                                  Total = 2,028,000 events/hour
                                        =       563 events/second

Payload = 563 ev/s × (1 KB metadata + 30 KB snapshot)
        = 563 × 31 KB ≈ 17.5 MB/s ≈ 140 Mbps  (peak, statewide, with imagery)
Metadata only (degraded mode) = 563 × 1 KB ≈ 0.56 MB/s ≈ 4.5 Mbps
```

Design	Sustained WAN load	Ratio
Central video ingest (Model 4 full)	**160 Gbps**	1×
Sentinel, events + snapshots	**~140 Mbps**	**~1,140× less**
Sentinel, metadata-only degraded mode	**~4.5 Mbps**	~36,000× less

2.3 The real WAN consumer is live viewing, not analytics

Analytics is cheap; operators watching video is not.

```
100 concurrent operators × 16 tiles × 2 Mbps = 3.2 Gbps
```

Mitigations, all implemented at the edge relay:

- **Adaptive viewing profile** — remote viewers get a 720p / 500 kbps transcode, not the source stream → 3.2 Gbps becomes **800 Mbps**.
- **Only visible tiles stream** (scrolled-off tiles pause) → typically halves it again.
- **Snapshot-first UI** — the map and alert rail use still images; video starts only on explicit intent.
- **Local viewing stays local** — a district operator watching district cameras never traverses the state backbone.

2.4 Low-bandwidth and disconnected operation

Link quality	Behaviour
Good (≥ 10 Mbps)	Full: events + snapshots + on-demand clips + live view
Constrained (1–10 Mbps)	Snapshots downscaled to 8 KB; clips deferred to an off-peak window; live view 360p on request only
Poor (< 1 Mbps)	**Metadata-only mode** — 1 KB events; snapshots retained locally with a pointer; retrieved only when an alert fires
Offline	**Edge autonomy**: analytics continues locally against a cached watchlist; events queue in a durable store-and-forward buffer (72 h default) and drain on reconnect, alerts prioritised ahead of routine events

Edge autonomy is a deliberate design claim: **a district node with a severed WAN link still reads plates, still matches the cached watchlist, still records evidence, and still alerts locally.** Nothing about the architecture makes policing depend on the backbone being up.

---

3. Compute and GPU sizing

3.1 Per-frame cost model

Measured/estimated on an NVIDIA L4 (TensorRT, FP16, batched), for the full ANPR chain:

```
vehicle detection (640², YOLO-class)      ≈ 2.5 ms/frame
plate detection on vehicle crops (×1.5)   ≈ 1.2 ms/frame
rectification + OCR on plate crops        ≈ 1.0 ms/frame
tracking + post-processing (CPU)          ≈ negligible on GPU
                                  Total ≈ 4.7 ms GPU per analysed frame
⇒ sustained capacity ≈ 300 analysed frames/second per L4 (at ~70% utilisation headroom)
```

What we have measured, rather than estimated: the same chain on a laptop CPU (base Apple M2, ONNX Runtime, no GPU) sustains about **16 analysed frames per second** on dense traffic, end to end through the real worker, with the plate detector the limit at about 30 plate reads per second (`tier capacity`). The L4 figure above is the vendor-benchmark estimate this plan sizes on; the laptop figure is what the on-site proof of concept runs on.

3.2 GPUs required

```
Tier A:  9,600 cams × 10 fps   =  96,000 fps ÷ 300 = 320 L4
Tier B: 22,400 cams ×  2 fps   =  44,800 fps ÷ 300 = 149 L4
Tier C: 28,000 cams × 0.3 fps  =   8,400 fps ÷ 300 =  28 L4
                                          Total ≈ 497 L4  (≈ 560 with N+1 redundancy)
```

≈ **140 servers with 4× L4 each**, distributed across 34 districts weighted by camera density (Ahmedabad and Surat carry a disproportionate share; several districts need a single 2-GPU node).

Decode is the co-constraint, and it is usually the one people forget. Each L4 has NVDEC engines good for roughly 40–50 concurrent 1080p30 decodes. Because we decode at the analytic rate rather than full rate — and because motion gating means tier C cameras are usually not decoded at all — decode fits inside the same node budget. Where it does not, decode-only CPU nodes with Quick Sync are added; this is called out as a sizing input rather than assumed away.

3.3 Edge box option for remote sites

For sites where backhaul cannot carry even the viewing profile, an on-site **Jetson Orin NX 16 GB** handles 8–12 cameras at tier B. Recommended for ~1,500 remote locations rather than statewide, because district-level GPU servers are roughly 3× cheaper per camera. The architecture supports both because the edge agent is the same code on both.

3.4 Core plane sizing

563 events/s is a modest stream — the core is dominated by storage and query, not throughput.

Component	Sizing
Kafka	3 brokers, 8 vCPU / 32 GB / NVMe, ~2 MB/s ingress, 7-day retention
Correlator / alerting	6–10 pods, horizontally scaled on consumer-group lag
PostgreSQL + PostGIS + TimescaleDB	Primary 32 vCPU / 256 GB / NVMe + 2 replicas, Patroni HA, PgBouncer; detections hypertable partitioned daily with continuous aggregates
OpenSearch (at this scale, now justified)	6 data nodes for detection search; Postgres remains system of record
Core API	8–20 pods behind an ingress, HPA on RPS
Object storage	Ceph or S3, erasure-coded, cross-site replication

---

4. Storage

4.1 What full centralisation would cost (the rejected option)

```
Per camera per day = 2 Mbps ÷ 8 = 0.25 MB/s × 86,400 s = 21.6 GB/day
Statewide          = 80,000 × 21.6 GB = 1.73 PB/day
7-day hot tier     = 12.1 PB      15-day = 25.9 PB      30-day = 51.8 PB
```

12.1 PB of hot, always-writable storage for a one-week window — before redundancy, before DR. At cloud object pricing this is of the order of **₹20+ crore per month**; on-premises it is a multi-hundred-crore build with its own data-centre footprint. **Rejected, and explicitly so in the presentation.**

4.2 What Sentinel stores

Data class	Volume	Retention	Tier
Detection events (metadata)	48.7 M/day × ~300 B ≈ **14.6 GB/day**	1 year online, then archive	Hot NVMe → warm
Evidence snapshots	48.7 M/day × 30 KB ≈ **1.46 TB/day**	7 days all; tier-A and watchlist-related kept 90 days	Hot 7 d → warm → purge
Alert clips (event-driven)	~24,000/day × 6 MB ≈ **146 GB/day**	90 days; case-linked clips 1 year+	Warm → cold
Continuous recording, **critical cameras only** (2,000)	2,000 × 10.8 GB (H.265) ≈ **21.6 TB/day**	15 days	Warm (erasure-coded)
Audit log	< 1 GB/day	5 years (statutory)	Warm, immutable, hash-chained

```
Steady-state footprint
  events (1 yr)                    ≈   5.3 TB
  snapshots (7 d hot + 90 d tier-A) ≈  10.2 TB + ~40 TB
  alert clips (90 d)                ≈  13.1 TB
  critical continuous (15 d)        ≈ 324.0 TB
                           Total  ≈ 390 TB usable  (≈ 600 TB raw with EC + DR)
```

390 TB versus 12.1 PB — a ~31× reduction — while preserving record, playback, scrub, export and chain-of-custody for everything that actually matters forensically. Historical routine footage remains in each department's existing 7–15 day retention and is retrievable on demand through the federation layer, which is precisely the "reuse existing infrastructure" mandate.

4.3 Lifecycle

`Hot (NVMe, 0–7 d)` → `Warm (HDD/EC, 7–90 d)` → `Cold (object/archive, 90 d–1 yr+)` → `Purge`, driven by a policy engine with per-data-class rules, legal-hold override for case-linked evidence, and a visible retention countdown in the admin UI.

---

5. High availability, DR and failure domains

Domain	Failure	Behaviour
Camera	Offline/tampered	Health alert within 30 s; registry marks degraded; gap-analysis map updates
Edge node	Crash	Local supervisor restart; peer node in the district picks up cameras via lease; buffered events replay
WAN link	Severed	Edge autonomy for 72 h; prioritised drain on reconnect
Core service	Pod failure	K8s reschedules; consumers resume from committed offsets
Database	Primary loss	Patroni automatic failover to a synchronous replica
Data centre	Site loss	Warm standby DC with streaming replication + object replication

Targets: RPO ≤ 15 min for central state (0 for edge-buffered events, which are durably queued locally), RTO ≤ 1 h for full service, RTO ≤ 5 min for alerting. Quarterly DR drills with documented runbooks; automated backups with restore verification, because an unverified backup is a rumour.

Drilled, not just planned. `make dr-drill` backs up the live database, restores it into a fresh one, compares every table's row count and the migration head, verifies the audit hash chain on the restored copy, then tampers with one row to prove verification catches it. Its first run failed (TimescaleDB hypertables need `timescaledb_pre_restore()`), which is the point of a drill. Measured: 1,000,000 detections restore in 5.7 s on one M2 core, so a logical restore covers the small, critical tables in seconds while the 1-hour RTO for a year of detections rests on replica failover and physical point-in-time restore. See `evidence/DR-RESTORE-DRILL.md`.

---

6. Indicative cost (₹)

Order-of-magnitude, for comparison of architectures rather than for tendering.

Line	Capex
District GPU analytics servers (140 × ~₹20 L, 4× L4)	₹28 cr
Remote edge boxes (1,500 × ~₹70 k)	₹10.5 cr
Core DC: compute, 600 TB storage, network, HA	₹14 cr
DR site (warm standby)	₹7 cr
District uplink upgrades (assumes GSWAN reuse)	₹9 cr
Software licences	**₹0 — fully open-source stack**
Integration, migration, training, rollout services	₹12 cr
Total capex	**≈ ₹80 cr**

Line	Opex / year
Power, cooling, DC space	₹5 cr
Network/bandwidth	₹7 cr
AMC + support (≈12% of hardware capex)	₹8 cr
Operations team (NOC, L1–L3)	₹4 cr
Total opex	**≈ ₹24 cr/yr**

For contrast, a full central-VMS build (Model 4 at 80,000 cameras) requires ~160 Gbps of backbone and 12 PB of hot storage for a single week of retention — conservatively **₹400 cr+ capex** with a materially higher opex, for capabilities that this design already provides where they matter. The open-source stack also removes per-camera VMS licensing, which at 80,000 cameras is typically a larger line item than the hardware.

---

7. Phased rollout

Phase	Duration	Scope	Exit criteria
0 — Pilot	0–3 months	1 district, ~500 cameras, 2 departments	ANPR accuracy validated on live feeds; alert latency < 5 s; operator acceptance
1 — Multi-department	3–9 months	4 districts, ~5,000 cameras, 6 departments, adapters for the top 3 VMS vendors	Federation proven across vendors; VAHAN/eGujCop integration live; DR drill passed
2 — Statewide core	9–21 months	All 34 districts, ~30,000 cameras, 15 departments	Statewide vehicle tracking operational; SOC/NOC established; training delivered
3 — Full scale + private	21–36 months	80,000 cameras + consented private camera onboarding programme	Full coverage; policy framework and audit regime institutionalised

Each phase is independently useful. The system delivers operational value at 500 cameras and never requires a big-bang cutover — which is the only kind of government rollout that actually finishes.

---

8. Prerequisites and information required from departments

The problem statement explicitly asks what is needed for feasibility. This is the data-collection template Sentinel requires per department — and it is itself a deliverable, because **no such consolidated inventory currently exists, and building it is Model 1's real value.**

Field	Why it is needed
Camera count, make, model, firmware	Adapter selection, ONVIF profile support
Analog vs IP; encoder model where analog	Determines whether a site needs an encoder or an edge box
VMS/NVR vendor, version, API availability	Federation adapter selection (Model 3)
Codec, resolution, frame rate, bitrate per site	Decode sizing, ragged batching configuration
Current retention period and storage location	What we do *not* need to re-store centrally
Network: link type, bandwidth, static IP/NAT/firewall posture	Tier assignment, relay placement, VPN/mTLS design
Geo-coordinates, mounting height, viewing direction, FOV	GIS registry and genuine coverage gap analysis
AMC vendor and contract end date	Rollout sequencing; avoids touching cameras under a live AMC
Nodal officer, escalation contact	Onboarding and incident routing
Data-sensitivity classification, access policy	RBAC scoping, retention and masking defaults

Assumptions stated openly (each with a mitigation, because a jury trusts stated assumptions far more than confident silence): GSWAN or equivalent connectivity is available to district headquarters; departments will permit read-only access to existing streams; a state data centre and DR site are available; VAHAN/SARTHI/eGujCop/AFIS/NAFIS integration will be granted through documented APIs; private camera onboarding will be governed by a consent and policy framework issued by the Home Department.

---

9. Capacity planning formulas (so the numbers can be re-derived)

```
WAN_analytics (Mbps)   = Σ_tiers (cameras × events_per_hour) ÷ 3600 × payload_KB × 8 ÷ 1000
GPUs_required          = Σ_tiers (cameras × analytic_fps) ÷ per_GPU_fps_capacity × (1 + redundancy)
Storage_hot (TB/day)   = events_per_day × (metadata_B + snapshot_B) ÷ 10¹²
                         + critical_cameras × bitrate_Mbps × 10,800 ÷ 10⁶
Viewing_WAN (Mbps)     = concurrent_operators × tiles × profile_Mbps × visible_fraction
Central_video (Gbps)   = cameras × bitrate_Mbps ÷ 1000        ← the number we refuse to pay
```

Change one input — camera mix, event rate, retention — and the whole model re-derives. That transparency is the point: it is a sizing model the department can own, not a vendor's assertion.

Next: 05-DELIVERY-PLAN — the build sequence.

05 — Delivery Plan

Sentinel · Build sequence for a solo full-time developer

> Assumption: ~6 hours/day, ~40 hours/week, single developer, deadline extended. Sequence is ordered by **risk retirement first**, then by scored value. Every milestone ends in something demo-able — if you stopped at any milestone, you would still have a coherent story.

> **Planning-figure correction (M15).** Figures of "~50 cameras" and references to a `GET /api/ingest` catalogue endpoint in this document date from planning, before the grid was accessible. The real grid assigned to us has **30 cameras**, and our host serves its catalogue at **`GET /cameras.json`** behind a session-cookie login — the integrator guide's `/api/ingest` path 404s there, because that guide is a shared template with `<host>` placeholders. The planning text is left as written rather than quietly rewritten, so the record of what was assumed stays visible; where this document states a present-tense fact, trust the README and `evidence/`.

---

1. Sequencing principle

The two things that can kill this project are not features. They are:

1. **Can one M4 sustain ~50 concurrent streams with ANPR?** — unknown until measured.
2. **Is open-source ANPR accurate enough on Indian plates at CCTV angles?** — unknown until measured.

Both are answered in **week 1**, before a single screen is designed. Everything after that is execution against known constraints. Building features on top of an unmeasured assumption is how hackathon projects die at 2 a.m. the night before submission.

---

2. Milestones

M0 · Foundations — days 1–2

- Monorepo scaffold (`apps/`, `services/`, `packages/`, `infra/`), `uv` workspace, ruff + mypy + pytest, pre-commit hooks
- `docker compose up` brings up Postgres/PostGIS/TimescaleDB, Valkey, NATS, MinIO, MediaMTX
- `sentinel_core`: event schemas (Pydantic v2), config, structured logging, OpenTelemetry bootstrap
- CI: lint + type + test on push
- **Exit:** `docker compose up` works from a clean clone; `pytest` green

M1 · Capacity truth — days 3–5 ⚠️ *highest-risk milestone, do it first*

- Catalogue client for `GET /api/ingest`; per-camera property mapping
- Capture harness: RTSP-over-TCP, VideoToolbox hardware decode, `StreamClock` (PTS-only), backoff supervisor, loop-cut detector
- **Local synthetic RTSP grid** (MediaMTX looping mixed-codec, mixed-resolution samples) so you can develop and chaos-test without touching the government gateway
- Benchmark script: N cameras × M fps until degradation, on M4 and M2
- **Exit:** a published capacity curve — "M4 sustains X cameras at tier A, Y at tier B" — and a tiering configuration derived from it. This number shapes every later decision and belongs in the HLD.

M2 · ANPR accuracy truth — days 6–10 ⚠️ *second-highest risk*

- Implement the full chain: vehicle detect → track → plate detect → rectify → OCR → temporal vote → format validation
- Export models to ONNX; CoreML EP on Mac, CUDA/TensorRT path documented
- Build a labelled evaluation set (300–500 plate instances captured from the test grid, hand-labelled)
- Measure plate-level and character-level accuracy; publish a benchmark table per pipeline variant
- **Decision gate:** public research shows **no credible pretrained Indian ANPR weights exist**, and the global OCR model's region list does not include India — so treat fine-tuning as *probable, not contingent* and budget 1–2 Colab days here. If plate-level

accuracy on legible plates is ≥ 85% off-the-shelf, defer the fine-tune and revisit after M9.
- **Exit:** accuracy report committed to the repo, with failure-mode examples. This table goes straight into the PPT — very few entrants will have one.

M3 · Registry + GIS (Model 1, mandatory) — days 11–14

- Schema: departments, sites, cameras (PostGIS point + FOV polygon), stream profiles, health
- Onboarding: catalogue auto-onboard, bulk CSV with column mapping and dry-run, manual entry, REST API
- Reconciliation job (catalogue diff → auto-update)
- Coverage gap analysis query + exportable report
- **Exit:** all ~50 test cameras onboarded automatically and visible on a map with live status

M4 · Detection pipeline end-to-end — days 15–19

- Edge agent: tiered frame scheduler, motion gating, ragged batching, backpressure
- Events published to NATS → persisted to TimescaleDB with snapshots in MinIO
- Health telemetry stream; Prometheus metrics; ops endpoints
- **Compliance test suite** mirroring the official checklist, one test per line
- **Exit:** run the whole test grid for an hour and produce a plate/timestamp CSV — this is already 80% of submission deliverable (4)

M5 · Watchlist, correlation, alerts — days 20–24

- Watchlist schema + matching ladder (exact → ambiguity class → edit distance → partial → attribute → ReID)
- Continuous correlation against a hot in-memory index; retro-scan on new entry
- Alert orchestrator: priority scoring, dedupe windows, lifecycle, escalation
- WebSocket alert stream
- **Exit:** add a plate to the watchlist → alert appears in a CLI/WS consumer within 2 s of the detection PTS, with a retro-scan of history

M6 · Design system + Operator Console core — days 25–32

- Tokens, primitives, component gallery at `/dev/gallery`
- Shell, navigation, auth, ⌘K palette
- **Home** (map + alert rail + health pill) and **Find a Vehicle** — the two screens that carry the demo, built to a higher standard than everything else
- **Exit:** type a plate → see the route on a map. The product becomes real at this milestone.

M7 · Live wall + playback — days 33–38

- WHEP player with HLS fallback, visible-tile-only streaming, transport/latency badge
- Adaptive grid, saved layouts, detection overlays
- Event-clip recording, sealed + hashed; timeline scrubber playback for recorded cameras
- **Exit:** 16 tiles live without dropped frames; click an alert → watch the sealed clip

M8 · Route reconstruction + cross-camera ReID — days 39–43

- Sighting → route assembly, spatio-temporal feasibility gate, appearance-embedding bridges
- Route replay animation, evidence strip, Movement Report export (PDF + CSV + hash manifest)
- **Exit:** the full BOLO moment, end to end, rehearsable in 90 seconds

M9 · Alerts screen, Cameras screen, Health/Ops — days 44–48

- Alert inbox and detail panel with the full triage workflow and "Not this vehicle" feedback loop
- Camera onboarding wizard with live preview; gap-analysis heat layer
- Health board + **Integrator Compliance panel**
- **Exit:** every Must-have feature is reachable through the UI

M10 · Admin Portal + Field PWA — days 49–54

- Admin: users/roles with permission preview, watchlist management, retention policies, audit log with "Verify integrity", integration cards

- PWA: alerts, plate lookup, report-a-sighting with offline outbox, nearby cameras; Web Push; installable
- **Exit:** three surfaces, one design system

M11 · Federation adapters + government integration stubs — days 55–59

- ONVIF driver (discovery, media profiles, PTZ, events); driver conformance suite
- Vendor shims (Milestone/Genetec/Hikvision/Dahua) against mock servers, with the plugin SDK documented
- `ExternalRegistry` providers for VAHAN/SARTHI/eGujCop/AFIS/NAFIS with conformant mocks and published contracts
- **Exit:** "onboarding department 27 is a driver, not a release" — demonstrated, not asserted

M12 · Security, privacy, hardening — days 60–63

- OIDC/RBAC/ABAC, Postgres RLS department tenancy, mTLS between edge and core
- Edge-side default face blurring; reveal-on-authorisation with reason capture
- Hash-chained audit log + verification CLI; retention policy engine
- `pip-audit` / `npm audit`, SBOM, secret scanning, signed media URLs
- **Exit:** a security section in the HLD backed by running code

M13 · Secondary analytics — days 64–67

- Vehicle class/colour/coarse-make; zone rules (intrusion, loitering, wrong-way, stopped vehicle); camera tamper detection
- *(No face recognition — excluded by decision; see 00-SOLUTION-PLAN §9. The freed time goes into vehicle attributes, which is also what cross-camera ReID needs.)*
- **Exit:** "reliable analytics beyond ANPR" — an explicit bonus criterion

M14 · Evidence run + deliverables — days 68–74

- Overnight multi-hour run across the full grid; auto-generated evidence report
- Record demo video (1): own feed, 2–3 min, scripted
- Record demo video (2): government feed + output report
- Write the PPT; export the HLD to PDF; polish README with screenshots and a 5-minute quickstart
- **Exit:** every submission artefact exists

M15 · Rehearsal, hardening, buffer — days 75–80

- Chaos drills: kill feeds, block 8554, force loop cuts, throttle the network — during a rehearsed demo
- Fresh-clone test on the M2 (proves the quickstart actually works on a machine that is not yours)
- Link verification in incognito; consistency pass across PPT/HLD/videos/repo
- Rehearse the three winning moments until they are muscle memory
- **Exit:** submitted, with days to spare

---

3. Weekly rhythm

Day	Practice
Every day	End with a working build. Commit early, commit often, meaningful messages — the repo is a scored artefact and its history is read.
Every Friday	Record a 2-minute progress video. By submission you will have a highlight reel and you will have rehearsed the narration a dozen times.
Every Sunday	Re-read the seven evaluation criteria. Score yourself out of 10 on each. Work next week on the lowest number.
Continuously	Keep an `evidence/` folder: benchmark CSVs, accuracy tables, screenshots, capacity curves. Claims backed by artefacts win; adjectives do not.

---

4. Where to use AI assistance aggressively

You are backend/AI-heavy and solo. Spend your own hours on the parts where judgement matters and delegate the rest:

Delegate heavily	Do yourself
React screen scaffolding from the specs in 03-UX-DESIGN	The ANPR pipeline and its accuracy tuning
CRUD endpoints, Pydantic schemas, migrations	The PTS/clock design and stream resilience behaviours
Test scaffolding, fixtures, mock servers	The matching ladder and route feasibility logic
PDF/CSV report generators	The scale model and the numbers you will defend live
Documentation formatting, diagram boilerplate	The demo narrative and the three winning moments

---

5. Definition of done (per feature)

A feature is done when: it works against the live test grid (not only the synthetic one) · it has tests · it degrades sensibly when its dependency is down · it appears in the UI with plain-language copy · it is covered by an evidence artefact (screenshot, CSV or metric) · and it is documented in one paragraph in the README.

Next: 06-SUBMISSION-PLAYBOOK — demo scripts and the evaluation-day checklist.

06 — Submission & Evaluation Playbook

Sentinel · Demo scripts, artefacts and the day-of checklist

---

1. The four required submissions

#	Requirement	Artefact	Owner milestone
1	Solution Presentation (PPT/PDF)	`deliverables/presentation/sentinel-solution.pdf`	M14
2	Technical Proposal / HLD	`docs/01-ARCHITECTURE.md` + `docs/04-SCALE-PLAN.md` → `deliverables/hld/sentinel-hld.pdf`	M14
3	Demo on your own feed (2–3 min)	Unlisted YouTube + Drive mirror	M14
4	Demo on government feed + output report	Unlisted YouTube + Drive mirror + `deliverables/reports/govt-feed-evidence.pdf` / `.csv`	M14

Plus: a **source archive** (the repository stays private — see 00-SOLUTION-PLAN §9) containing a README a stranger can run in five minutes, and a `SOURCE-TOUR.md` guided reading path.

---

2. Presentation outline (14 slides, no more)

1. **The problem in one picture** — 26 islands, 80,000 cameras, a getaway car that requires phone calls to trace
2. **What we built** — one sentence and one screenshot of the route-replay screen
3. **Model choice and justification** — Hybrid 1+2+3+selective-4, and *why each*
4. **The rejected option, with numbers** — 160 Gbps, 12.1 PB/week, ₹400 cr+. The honesty moment.
5. **Architecture** — the one diagram, layered, readable from the back of the room
6. **Ingest & federation** — heterogeneous sources, adapter framework, zero change to department systems
7. **ANPR pipeline** — stages, and the accuracy table on live government feeds
8. **Watchlist correlation** — the matching ladder, with the explainability screenshot
9. **Alerting & triage** — priority, dedupe, escalation, false-positive feedback loop
10. **Cross-camera tracking** — the GIS route with confirmed vs probable hops
11. **Scale to 80,000** — the tiering table, GPU/bandwidth/storage math, phased rollout
12. **Security & privacy** — default blur, reveal-on-authorisation, hash-chained audit, DPDP alignment
13. **Operational impact** — minutes-to-trace before vs after; what a constable can now do unaided
14. **PoC readiness** — what is running today, what we need from departments, the 90-day pilot plan

Design rules: one idea per slide · every number sourced · no clip-art · screenshots of the real product, never mock-ups · dark slides matching the product's visual identity, so the deck and the demo feel like one thing.

---

3. Demo video 1 — own feed (target 2:45)

Rewritten in M15 against the system as built. Every command below was run; every number is one this platform actually produced. The earlier draft of this section was written before implementation and referenced things that do not exist — see §4's note on the catalogue endpoint for the sharpest example.

Before recording. Clean desktop, notifications off, 1080p60. Start the stack and let the worker settle for a minute so the map is populated:

```
make up && make api          # :18000
make web                     # :5173
uv run python scripts/synthetic_grid.py all
make worker                  # synthetic grid
```

Sign in as `admin@sentinel-platform.com` / `sentinel-admin-2026` (the local development login; never used outside a developer machine — see 10 — Finale Runbook §3).

Time	Beat	On screen
0:00–0:15	"This is Sentinel, running on my machine. Nothing here is a mock-up."	Terminal: `make ps` → every service healthy. Cut to the console.
0:15–0:35	The fleet is real	Overview: cameras on the map, the status strip reading *N live · 0 down*. Hover a marker — it names the camera and its state.
0:35–1:05	Live ANPR	Live Wall, 2×2. Tiles are playing. Cut to the terminal running the worker: plate reads scrolling with confidence values. "Every timestamp comes from the stream's own clock, not when the packet arrived."
1:05–1:25	Arm the watchlist	Find a Vehicle → search a plate the worker has just read → *Watch for this vehicle* → mark it **Stolen**, add a case reference.
1:25–1:55	**The ambiguity moment**	Ingest a read of the same plate with OCR confusions (`8→B` , `0→O` , `5→S`). The alert still fires, tagged `ambiguity_class` . "An operator typed one plate. The camera read a different string. We still caught it."
1:55–2:25	Trace it	Alerts → the hit → Find a Vehicle → timeline with per-sighting confidence, map framed on the sightings, *Replay route*.
2:25–2:45	Close on evidence	*Export report* → the generated Movement Report PDF with its hash manifest.

The ambiguity beat is the one to rehearse. It is the single clearest demonstration that this is a real matching system rather than a string compare, and it takes one command:

```
uv run python - <<'EOF'
import httpx, uuid
from datetime import datetime, UTC
from sentinel_core.config import Settings
from sentinel_core.plates import normalise_plate, ambiguity_key
s = Settings(); variant = 'GJ1B67OS'          # watchlist holds GJ186705
httpx.post('http://localhost:18000/api/v1/detections',
    json={'event_id': str(uuid.uuid4()), 'camera_id': 'dev-cam-01',
          'observed_at': datetime.now(UTC).isoformat(), 'plate_text': variant,
          'plate_normalised': normalise_plate(variant),
          'plate_ambiguity_key': ambiguity_key(variant),
          'plate_confidence': 0.68, 'frames_voted': 4, 'vehicle_class': 'truck',
          'vehicle_colour': 'blue', 'pts_ms': 5678.0, 'node_id': 'demo',
          'model_versions': {'demo': '1'}},
    headers={'X-Service-Token': s.edge_service_token.get_secret_value()})
EOF
```

Production notes: narrate calmly, no music, subtitles burned in (juries often watch muted). Show a terminal at least twice — it is the cheapest proof that a backend exists.

---

4. Demo video 2 — government feed + report (target 3:30)

> **Correction worth knowing before you record.** The integrator guide's generic `GET /api/ingest` path **404s on our assigned deployment** — that guide is a shared template with `<host>` placeholders, and this deployment only implements `GET /cameras.json` , behind a session-cookie login. Saying "/api/ingest" on camera would be saying something untrue about our own integration. See `evidence/M4-GOV-CAMERA-GRID-INTEGRATION.md` .

> **Pace the run.** The sandbox enforces an account-level viewing-time quota and returns `403 watch time limit reached`, cutting every feed. Do not leave a large run going before recording. Six cameras is a comfortable number.

Time	Beat	On screen
0:00–0:25	Discovery is not hard-coded	Terminal: authenticate, `GET cameras.json` → 30 cameras with real Gujarat names. Then one call to `POST /api/v1/cameras/discover` → **30 onboarded, 0 failed**.
0:25–0:50	The grid on a map	Overview: real cameras plotted across Ahmedabad, Junagadh, Rajkot, Navsari. Note openly that 21 of 30 geocode from their names and the rest are deliberately left unplotted rather than guessed.
0:50–1:25	Live government video, no credentials in the browser	Live Wall playing real traffic. Open devtools: the video URL is our own `localhost` relay path. "The camera's credentialed URL never reaches the browser."
1:25–2:00	ANPR on real traffic	Worker terminal: reads scrolling from `cam01…cam06`. Detections table filling. Cite the measured run: **172 detections, 124 distinct plates in ~10 minutes across 6 cameras**.
2:00–2:25	Beyond plates	Cameras table showing a camera flagged **`moved`** by tamper detection, and zone events — **186 intrusion + 22 wrong-way** on real footage.
2:25–2:50	It survives the network	Run `uv run python scripts/chaos_drill.py --drill feed-loss` live. The camera drops, the platform notices, reconnects and recovers unattended.
2:50–3:10	Compliance, checked not claimed	Admin → Integrator Compliance: green ticks. "These are assertions in a test suite, not a slide."
3:10–3:30	The report	`scripts/evidence_report.py` → open `evidence/M14-EVIDENCE-RUN.md` and the CSV. End on the findings document: the quota, the double-connection issue, and `evidence/ANPR-ACCURACY.md`: an accuracy number we measured ourselves, below our own bar, with its limits stated.

Ending on the findings document is a deliberate choice. A jury that has sat through ten decks of unbroken success will remember the team that showed its own measured limitations — and it is the honest thing to do.

Output report (`evidence/M14-EVIDENCE-RUN.md` + `m14-detections.csv`): run window, fleet composition, throughput, read-quality distribution, secondary analytics, resilience counters, ingest latency, and every detection row (plate, confidence, camera, class, colour). Generated by script, not by hand.

---

5. Source archive presentation

The repository is private, so the archive you hand over *is* your code credibility. It will be skimmed, not studied — optimise for the skim.

- `README.md`: one-paragraph pitch, a hero screenshot, a 90-second architecture summary, **a five-minute quickstart that actually works on a clean machine**, a feature list mapped to the seven evaluation criteria, and a link to these docs
- **`SOURCE-TOUR.md`** — a guided five-minute reading path: start at the edge agent's capture loop, then the ANPR pipeline, then the matching ladder, then the alert orchestrator. Name the file and the function for each stop. This is the single highest-leverage page in a private submission.
- `docs/` as published here; `evidence/` with benchmark CSVs, the accuracy table and the capacity curves
- Clean commit history with meaningful messages — evidence of sustained engineering rather than a weekend paste
- No secrets, no model weights committed (download scripts instead), no dead code, no `TODO: fix this` in a path a reviewer will open
- LICENSE file, and a `THIRD-PARTY-LICENCES.md` listing every dependency and its licence — the challenge mandates open-source technologies, and showing you tracked licences (including the AGPL components and why they are acceptable) is exactly the diligence a government buyer looks for
- CI configured and green; include the workflow file even though the badge is not publicly visible

Consider before you upload: the challenge mandates open-source technologies. A fully closed submission can read as being in tension with that spirit. If you can make the repository public at submission time without losing anything you actually care about, it is worth doing — it is a one-click change and it converts an explanation into an asset.

---

6. Pre-submission checklist

Organisers' technical checklist — verified by the automated compliance suite, and screenshotted for the PPT:

- ☐ All clients force RTSP over TCP
- ☐ No logic depends on declared FPS or arrival time
- ☐ Inter-frame gaps do not crash the pipeline
- ☐ Backoff reconnect tested by restarting a feed mid-run
- ☐ Decoder warnings logged, never fatal
- ☐ Camera discovery is catalogue-driven
- ☐ Mixed codec and resolution handled
- ☐ Sane behaviour across the loop cut

Submission hygiene:

- ☐ YouTube videos set to *Unlisted* and verified in an incognito window
- ☐ Drive/OneDrive mirrors set to "Anyone with the link — Viewer", verified in incognito
- ☐ Source archive uploaded (or private link with reviewer access granted and verified from a second account)
- ☐ `SOURCE-TOUR.md` and `THIRD-PARTY-LICENCES.md` included in the archive
- ☐ Names, dates and version numbers consistent across PPT, HLD, videos and source archive
- ☐ PDFs open correctly and are under any portal size limits
- ☐ Every claim in the PPT is demonstrable in the videos — no orphan claims
- ☐ Contact details correct; category selected correctly at registration
- ☐ Uploaded with days to spare, not hours

---

7. Evaluation-day playbook (Phase 2, live)

Before you start

- Two machines, both fully set up. The M2 is the hot spare, tested from a fresh clone.
- Local synthetic grid running as a fallback if the venue network blocks the real feeds.
- Recorded demo videos on local disk — if the network dies entirely, you still have a story.
- Wired network if possible; phone hotspot as backup.
- Pre-warm: services up, models loaded, cameras onboarded, caches hot. Never cold-start in front of a jury.

When they give you the vehicle number

1. Say what you are about to do before you do it — juries follow narration better than clicks.
2. Type the plate into the single search field. Do not open a settings panel. The simplicity *is* the demo.
3. Show the retro-search result first (route on the map), then point out that a BOLO is now armed forward in time.
4. Replay the route. Let it run — the animation does the explaining.
5. Open one hop's evidence: snapshot, camera, department, PTS-accurate timestamp, confidence.
6. Export the Movement Report and hand it over. Ending with an artefact in the evaluator's hands is worth more than another screen.

Questions you will be asked, and the short answers

Question	Answer
"What if the plate is dirty or partly unreadable?"	Show the ambiguity classes (search `GJ01AB1Z34`, get `GJ01AB1234`), then a one-character misread coming back as a dashed *probable* hop with the plate actually read. Be plain that wildcard search and appearance ReID are designed (01-ARCHITECTURE §6.1) but not built.
"How does this scale to 80,000?"	The tiering table and the three formulas. Offer to re-derive with their numbers.
"What about privacy?"	**"We do no face recognition at all — faces are detected only so we can blur them."** Then: default blur at the edge, reveal-on-authorisation with reason capture, hash-chained audit, retention engine, false-match redressal
"Will this disturb our existing systems?"	Read-only, pull-based, standards-first; no agent on department infrastructure; the department's VMS keeps running untouched
"How long to onboard a new department?"	A driver, not a release — show the conformance suite and the plugin SDK
"What do you need from us?"	The data-collection template in 04-SCALE-PLAN §8 — answer with a document, not a wish
"What doesn't work yet?"	Name it plainly. The Won't-do list in the master plan exists precisely so you can answer this without flinching.

Demeanour: calm, slow, specific. Say "I measured that" and show the artefact. When something breaks — and something will — narrate the recovery: *"That camera dropped; the supervisor is backing off and will reconnect in about eight seconds."* Then let it reconnect on camera. Handling a failure gracefully in front of a jury is a stronger signal than a flawless run, because it is the thing they cannot fake-check.

Back to 00-SOLUTION-PLAN.

07 · Federation Driver SDK

Companion to 01-ARCHITECTURE.md §4.1. This document exists because M11's exit criterion is "onboarding department 27 is a driver, not a release — demonstrated, not asserted." What follows is the demonstration: a real interface, two real drivers against it, four contract-shaped vendor shims proving the pattern generalizes, and a conformance suite that runs the same contract against all six.

---

1. The port

Every camera source — no matter the protocol or vendor — implements one `Protocol`:

```
# packages/sentinel_core/src/sentinel_core/drivers/base.py
class CameraSource(Protocol):
    driver_id: str
    capabilities: SourceCapabilities

    async def discover(self, scope: DiscoveryScope) -> list[CameraDescriptor]: ...
    async def open(self, camera: CameraDescriptor, profile: StreamProfile) -> StreamHandle: ...
    async def health(self, camera: CameraDescriptor) -> SourceHealth: ...
```

- `discover` finds cameras — either a directed probe at one known host, or (protocol permitting) a broadcast/multicast scan of a network.
- `open` resolves a stream profile into something a player/relay can actually consume (a URL + protocol, today; nothing here prevents a future driver returning a live socket instead).
- `health` answers "is this camera reachable right now," independent of whether anything is actively streaming from it.
- `capabilities` is a `SourceCapabilities` value the UI reads directly to grey out actions a given camera's driver genuinely can't do (e.g. PTZ controls on a plain RTSP-only camera) — see docs/03-UX-DESIGN.md's capability matrix.

2. Drivers shipped

Driver	File	Status
`rtsp`	`services/edge_agent/src/edge_agent/adapters/rtsp_driver.py`	Real — implements `CameraSource`, wrapping the M1 capture pipeline (`pipeline/capture.py`) that has driven this project's synthetic grid since the beginning.
`onvif`	`packages/sentinel_core/src/sentinel_core/drivers/onvif.py`	Real — implements `CameraSource` via WS-Discovery (UDP multicast probe, with a directed-host fallback that's the reliable path on segmented CCTV networks), `GetDeviceInformation`, `GetProfiles` / `GetStreamUri`, and PTZ `ContinuousMove` / `Stop`. Hand-rolled SOAP/XML (no WSDL-based client library), with real WS-Security UsernameToken digest auth — not HTTP Basic, which most ONVIF devices reject.
(gov catalogue)	`packages/sentinel_core/src/sentinel_core/gov_catalogue.py`	Real, but **not** a `CameraSource` — a separate, simpler discovery-only client (list cameras + bulk-import) predating this Protocol, already wired into the onboarding wizard's "Connect a department system" step. Left as-is rather than force-fit into `CameraSource`, since it has no stream-open/health-check responsibility of its own (cameras it discovers are onboarded as plain `rtsp` sources).
`milestone` / `genetec` / `hikvision` / `dahua`	`packages/sentinel_core/.../drivers/{milestone,genetec,hikvision,dahua}.py`	Contract-shaped mocks — real protocol shape, proven against a mock server, not a licensed vendor SDK. See §7 below.

Onboarding through a driver. `onvif`, `hikvision` and `dahua` are exposed to operators through `POST /api/v1/cameras/discover-device` (admin only) and the wizard's **Discover a device** form: directed discovery against one address and account, a dry-run listing of every stream, then an upsert of each as a registry camera the edge worker's `--source registry` mode analyses. Drivers build stream URLs with `sentinel_core.stream_urls.with_credentials`, which percent-encodes the account — ONVIF's `GetStreamUri` returns credential-free URIs by spec, and a raw `@` in a password would otherwise redirect the connection to the wrong host.

3. Why ONVIF is hand-rolled SOAP, not a WSDL client library

Libraries like `python-onvif-zeep` generate their client from bundled WSDL files, which then need to be kept in sync with whichever ONVIF profile version a given camera implements. For the handful of operations this driver actually needs (device info, media profiles, stream URI, two PTZ calls), constructing and parsing the XML directly means:

1. No WSDL files to vendor and update.
2. The entire wire protocol is visible in one file — useful for a jury (or a future maintainer) who wants to see *exactly* what "ONVIF support" means here, not trust an opaque code-generation step.

3. WS-Security digest auth (`SHA1(nonce + created + password)` , base64) is implemented with real cryptographic primitives (`hashlib` , `secrets`), not stubbed — see `_wsse_header()` .

4. Testing without a physical camera

`packages/sentinel_core/tests/test_onvif_driver.py` runs the driver against an inlined mock ONVIF device (an `httpx.MockTransport` handler) that:

- Serves realistic canned SOAP responses for each operation.
- **Recomputes the WS-Security digest itself** and rejects a request whose password doesn't match — proving the auth path is exercised for real, not just "does the code run without crashing."

`services/edge_agent/src/edge_agent/adapters/rtsp_driver.py` reuses the same `RtspCapture` / `VideoSource` test-double pattern the capture pipeline's own tests already established (`FakeVideoSource`), via an injectable `backend_factory` constructor argument.

5. The conformance suite

`tests/test_driver_conformance.py` runs the **same five contract checks** against every driver in `_DRIVER_CASES` , parametrized:

1. `discover()` with a valid scope returns at least one camera with at least one stream profile that has a URL.
2. `discover()` with an empty/unreachable scope returns an empty list — **never raises**. (This caught a real bug during development: the ONVIF driver's multicast probe raised `OSError` on a network where multicast is unreachable, which is common on segmented CCTV networks. Fixed to degrade to an empty result instead of crashing — see the driver's `probe_multicast()` docstring.)
3. `health()` of a just-discovered camera reports `LIVE` .
4. `open()` returns a `StreamHandle` whose URL/protocol match the profile passed in.
5. `capabilities` is genuinely declared (not inherited/defaulted) per driver.

Adding driver #3 to this suite is one new `DriverCase(...)` entry — that line is the actual evidence for "a driver, not a release."

6. The vendor shims — proving the pattern generalizes

`docs/05-DELIVERY-PLAN.md` 's M11 exit criterion calls for "vendor shims (Milestone/Genetec/Hikvision/Dahua) against mock servers." Four `CameraSource` implementations exist for exactly this — deliberately at "contract + conformance-tested mock" fidelity, not a licensed vendor SDK integration (no SDK licence is used or implied for any of them):

Driver	Wire format	Auth	Why this one's useful to demonstrate
`hikvision.py`	ISAPI: plain HTTP + XML	RFC 7616 HTTP Digest (`httpx.DigestAuth`)	The most common "just a standard, no SDK needed" shape — a huge share of the deployed base speaks ISAPI even on non-Hikvision-branded rebadged cameras.
`dahua.py`	CGI: plain HTTP + flat `key=value` text (not XML/JSON)	HTTP Digest	A genuinely different response wire format from every other driver — proves the `CameraSource` port doesn't assume JSON/XML anywhere.
`milestone.py`	REST/JSON	OAuth2 password-grant, bearer token (`/IDP/connect/token`)	The enterprise-VMS shape: auth is a separate token exchange, not per-request credentials.
`genetec.py`	REST/JSON	Login call returns a session token, sent back as a custom `Session` header (not a cookie, not a bearer token)	A third distinct auth shape again — proves the port handles "however this vendor does it," not just one auth pattern.

Each shim is genuinely exercised end-to-end in `packages/sentinel_core/tests/test_vendor_shims.py` (vendor-specific edge cases — multi-channel discovery, disabled/no-stream cameras, wrong-password rejection) and in `tests/test_driver_conformance.py` (the shared five-check contract). The Hikvision/Dahua mocks **recompute the real digest response server-side** to verify a request, the same way `test_onvif_driver.py` 's mock does for WS-Security — auth is proven, not assumed.

All four are directed-discovery-only (`DiscoveryScope(host=...)` , `scope.host=None` returns `[]`): none of these protocols has a broadcast discovery mechanism the way ONVIF's WS-Discovery does, and in every real deployment of these VMS/NVR products the

operator already knows the Management Server or device IP being onboarded.

7. Adding a new driver (the "department 27" walkthrough)

1. Implement `CameraSource`'s three async methods for the new protocol/vendor SDK, plus `driver_id` and `capabilities`.
2. If there's no live device to test against, mock it: an `httpx.MockTransport` handler (SOAP/REST) or an injectable backend factory (SDK client) — whichever matches how the real thing talks over the wire. Both existing drivers show a worked example of each style.
3. Add one `DriverCase` entry to `tests/test_driver_conformance.py`. Passing the five shared contract checks is the bar for "this driver is real," independent of anything vendor-specific.
4. Register the driver's `driver_id` wherever cameras are onboarded (registry `CameraCreate.driver_id`, or catalogue-driven discovery) — no other part of the system needs to change; every consumer of a camera already only depends on `CameraDescriptor` / `StreamProfile`, not on which driver produced them.

08 · Security, Privacy & Hardening (M12)

Companion to 05-DELIVERY-PLAN.md's M12 milestone: "OIDC/RBAC/ABAC, Postgres RLS department tenancy, mTLS between edge and core; edge-side default face blurring with reveal-on-authorisation; a hash-chained audit log with a verification CLI; retention policy engine; dependency/secret scanning, SBOM, signed media URLs." Exit bar: "a security section in the HLD backed by running code" — this document, kept honest against what's actually implemented as each piece lands.

---

1. mTLS between edge and core

Decision: terminate at a reverse-proxy gateway, not inside core_api's own ASGI app. FastAPI/uvicorn (ASGI) has no standard extension for handing a verified peer client certificate up to application code — that's normally solved with a TLS-terminating gateway in front of the app, which is also the standard production pattern (a service mesh sidecar, a load balancer, or — as here — nginx), not a workaround.

What's real:

- `scripts/generate_mtls_certs.sh` mints a local CA, an edge-gateway server cert, and one client cert per edge node using real `openssl` commands — nothing here is a fake/placeholder PKI. Certs are gitignored; every developer/CI run mints their own.
- `infra/compose/edge_gateway/default.conf.template` — an nginx config (behind the `mtls` compose profile: `docker compose --profile mtls up -d edge-gateway`) that listens on `:8443`, requires a client certificate signed by the local CA (`ssl_verify_client on`), and proxies to core_api. **Verified**: a connection with no client cert is rejected at the TLS/ HTTP layer (`400 No required SSL certificate was sent`) before it ever reaches a request handler; a connection with a cert signed by a *different* CA is also rejected (`400 The SSL certificate error`); a connection with a valid cert is forwarded through to core_api, which responds exactly as it would to a direct call (proven by posting a real detection and camera-registration through the gateway end-to-end).
- nginx forwards the verified certificate's subject DN as `X-Client-DN`, plus a shared secret (`sentinel_core.config.mtls_gateway_shared_secret`) as `X-Edge-Gateway-Secret`. `core_api/security/mtls.py`'s `edge_gateway_identity` dependency only trusts the DN when the secret matches — a request that reaches core_api *directly* (bypassing the gateway, e.g. on the plain HTTP dev port) can't spoof an edge identity by just setting `X-Client-DN` itself, since it won't know the secret.
- This is **additive**, not a replacement for the existing `edge_service_token` bearer-token check the detection-ingest and camera-health endpoints already require (M6). The gateway adds network-layer admission control (must present a valid client cert to even open a connection); the service token remains the application-layer authorization check. Wired into one representative endpoint (`POST /api/v1/detections`) as an audit-trail enrichment — logged, not yet persisted to a durable table (that's section 3 below, once built).

Honest gap: the default `docker compose up` does *not* start the gateway (it's behind the `mtls` profile) — the plain HTTP dev port (`:18000`) remains reachable directly for local development ergonomics. A real deployment would firewall that port to localhost/the edge-gateway container only, making the gateway the sole path in from outside.

Verification, step by step:

```
./scripts/generate_mtls_certs.sh edge-node-01
docker compose -f infra/compose/docker-compose.yml --profile mtls up -d edge-gateway

# Rejected — no client certificate:
curl -sk https://localhost:8443/api/v1/admin/integrations
# -> 400 No required SSL certificate was sent

# Rejected — certificate not signed by our CA:
openssl req -x509 -newkey rsa:2048 -keyout /tmp/evil.key -out /tmp/evil.crt -days 1 -nodes -subj "/CN=evil"
curl -sk --cert /tmp/evil.crt --key /tmp/evil.key https://localhost:8443/api/v1/admin/integrations
# -> 400 The SSL certificate error

# Accepted — forwarded through to core_api:
curl -sk --cert infra/tls/client-edge-node-01.crt --key infra/tls/client-edge-node-01.key \
  https://localhost:8443/api/v1/admin/integrations
# -> {"detail":"Not authenticated"}  (reached core_api's own auth check — proves the proxy worked)
```

2. RBAC/ABAC + Postgres RLS department tenancy

Decision: real Postgres row-level security, enforced against a genuinely non-superuser role — not application-level `WHERE` clauses dressed up as "RLS." Postgres RLS policies are unconditionally bypassed for superusers and table owners, no exceptions, no config knob — that's a hard rule, not an oversight. The dev Postgres image's `sentinel` user is *both*. Making RLS mean anything therefore required a second, deliberately weaker role.

What's real:

- `infra/compose/init/02-app-role.sql` creates `sentinel_app`: `NOSUPERUSER NOBYPASSRLS`, granted ordinary table privileges. `sentinel_core.config`'s new `app_database_url` (vs the existing `database_url`, still used by Alembic migrations and by `sentinel` for DDL) is what `core_api/db/base.py`'s engine now actually connects as — **core_api's own request- handling queries run as a non-superuser role**, a real hardening step independent of RLS specifically (least privilege).
- Migration `23c60ee4c006` adds `users.department_id` (nullable FK) and enables **`FORCE ROW LEVEL SECURITY`** (not just `ENABLE` — `FORCE` is what stops the table *owner* from also bypassing it; the owner here is still `sentinel`, so this matters) on `cameras`, `detections`, `alerts`, with one policy per table keyed on the `app.current_department_id` session GUC.
- **Policy semantics, deliberately chosen for safety against the existing pre-M12 data**: `NULL` GUC (unset — no user logged in, or a user with no department of their own) means *unrestricted*; a set department scopes to that department's own cameras **plus any camera with no department assigned yet**. Every camera the synthetic dev grid has ever onboarded has `department_id IS NULL` — turning RLS on does not silently vanish the entire demo grid for anyone.
- `core_api/security/tenancy.py`'s `TenancyMiddleware` decodes the caller's JWT (which now carries a `department_id` claim — see `auth/service.py`'s `create_access_token` / `decode_access_token`) before any route dependency runs, and stashes it on `request.state`. `core_api/db/base.py`'s `get_session` reads that and issues `SET LOCAL app.current_department_id = '<uuid>'` for the request's first transaction — a middleware, not a per-route `Depends()`, specifically so every existing `Depends(get_session)` route gets this transparently, with no signature changes across the whole router surface.
- **Verified end-to-end through the real app**, not the ORM in isolation: `services/core_api/tests/test_rls_tenancy.py` creates two departments with one camera each and one operator per department, logs each in for a real JWT via `httpx.AsyncClient` + `ASGITransport` against the actual `create_app()`, and confirms: each operator's `GET /api/v1/cameras` returns only their own department's camera; the seeded (department-less) admin sees both; and `GET /api/v1/cameras/{other_depts_camera_id}` returns a plain **404**, not 403 — RLS is a `WHERE`-clause filter, so a hidden row is indistinguishable from a row that never existed, which is the correct behaviour (it doesn't even confirm the ID exists).

Honest gaps:

- `watchlist_entries`/alerts-by-plate search are **not** department-scoped — watchlists are modelled as a shared, statewide resource in this domain, not a per-department private list. This is a deliberate scoping choice, not an oversight.
- `SET LOCAL`'s scope is exactly one transaction. A route that calls `session.commit()` mid-handler and then issues further queries in the same request loses the restriction for anything after that commit — no current department-scoped *read* route does this, but it's a real sharp edge, documented rather than hidden.

- OIDC (an actual external identity provider) is not implemented — auth remains the M6 password-JWT login gate, now carrying a `department_id` ABAC attribute. Swapping in a real OIDC provider would replace the login endpoint's token issuance, not the RLS/tenancy machinery downstream of it, which only cares that a JWT carries a role and a department.

3. Edge-side default face blurring + reveal-on-authorisation

Decision: blur by default at the edge, keep the unblurred original in a separate directory only an explicit, reason-captured reveal can reach. Every ANPR event's snapshot is face-blurred before it ever leaves the edge node — there is no "unblurred by default" code path to accidentally hit.

What's real:

- `services/edge_agent/.../analytics/face_blur.py` — real face detection via OpenCV's YuNet (`cv2.FaceDetectorYN`), not a stub. **Discovered while building this**: the originally-planned classic Haar cascade (`cv2.CascadeClassifier`) doesn't exist at all in this repo's pinned `opencv-python-headless==5.0.0.93` — OpenCV 5.0 removed the entire legacy objdetect Haar API. YuNet (a small ~230KB ONNX DNN detector from the official OpenCV Zoo) is the modern replacement, downloaded once and cached on first use (same policy as models/README.md's other model weights — this one just doesn't have a wrapping Python package doing the download for it yet, so `ensure_model_downloaded()` does it directly). Runs over the **full frame**, not just the vehicle's bounding box — bystanders elsewhere in shot get the same default protection as the vehicle's own occupants.
- `services/edge_agent/.../analytics/snapshot_writer.py` — writes both artefacts every event needs: the blurred version (what `snapshot_uri` refers to, what core_api serves by default) and the true unblurred original, to two separate directories. **A blur failure never falls back to saving the unblurred frame as the "safe" default** — it writes nothing at all and the event still gets emitted with no snapshot, logged as an error. Verified: a real face (a public-domain test portrait) is genuinely, visibly blurred beyond recognition by this pipeline — not merely "a box is drawn," pixels are actually destroyed (Gaussian blur strong enough that re-running the detector on the blurred output finds nothing).
- `AnprPipeline` gained an optional `snapshot_writer` port (the same fake-able-port pattern as its vehicle detector/plate reader); wired into `edge_agent.worker` 's real camera loop, and `event.evidence.snapshot_uri` now actually flows through to `DetectionIn` (previously a schema field that existed end-to-end in the wire types but nothing ever populated — discovered while building this).
- `core_api` : `GET /api/v1/detections/{event_id}/snapshot` serves the blurred file to any authenticated user (no reveal needed — it's already the safe default). `POST /api/v1/detections/{event_id}/reveal-face` is **admin-only** (no separate "supervisor" role exists yet — a documented scoping choice) and requires a `reason` (min 5 characters, rejected with 422 otherwise) before it will even look at the unblurred original; every call is logged (actor, reason, timestamp) — genuinely reaching stdout thanks to the `configure_logging()` fix in section 1 above. `Detection.snapshot_uri` is an opaque `snapshot://<event_id>` marker, never a raw filesystem path — core_api resolves both the blurred and original file locations from `event_id` by its own configured directory convention, so nothing on the wire ever leaks a host path.
- **Verified end-to-end over real HTTP**: created a detection with a snapshot, confirmed `GET .../snapshot` returns the blurred file (200); confirmed `POST .../reveal-face` with an empty reason is rejected (422) and with a real reason returns the genuinely different unblurred file (200) while logging the audit line; confirmed an `operator` -role account can view the blurred snapshot but is rejected (403) from reveal, while `admin` can do both.

Honest gap: live, fully organic verification (a real synthetic-grid vehicle triggering ANPR, the pipeline blurring its actual captured frame, the resulting event flowing all the way to a browser) was not completed in this sandbox — ANPR vote-resolution didn't complete within several minutes of runtime here (confirmed this is pre-existing sandbox timing, not a regression: reproduced identically on the pre-M12 code). Every individual link in the chain was independently verified instead: real face detection/blur against a real photo, `SnapshotWriter` writing both files against the real model with no crash, the pipeline wiring via unit tests with a fake writer, and the full core_api HTTP surface (serve + reveal + role gate) against manually-seeded detection rows.

4. Hash-chained audit log + verification CLI + retention execution

Decision: a real append-only hash chain, not a plain timestamped log table. docs/01-ARCHITECTURE.md §7: "audit_log rows carry prev_hash/ row_hash — an append-only hash chain, verifiable by a CLI command."

What's real:

- `AuditLogEntry` (migration `c882d80a41a6`): `row_hash = SHA256(prev_hash + canonical_json(id, actor_email, action, resource_type, resource_id, detail, created_at))` . The first row chains from a fixed, documented genesis value (`"0" * 64`); every row after chains from the previous row's actual stored `row_hash` .

- `core_api/audit/service.py`'s `record_audit_event` serializes concurrent appends with a **Postgres transaction-scoped advisory lock** (`pg_advisory_xact_lock`) around the read-previous-hash-then-insert sequence — without it, two concurrent callers could both read the same "last row" and each compute a `row_hash` chained from the same `prev_hash`, silently forking the chain instead of extending it. A dedicated strictly-monotonic `seq` (Postgres `IDENTITY`) column — not the UUID `id`, and not `created_at`, which two rows could in principle share at whatever timestamp precision — is what "the previous row" unambiguously means.
- **Found and fixed a real bug while building this**: SQLAlchemy's ORM `default=` (as opposed to `server_default=`) is *not* evaluated at object construction — `AuditLogEntry(...).id` / `.created_at` are genuinely `None` until the object is flushed. Hashing before that first flush would have silently hashed the placeholder `None`s. Fixed with an explicit flush-then-hash-then-flush sequence.
- `verify_chain` walks every row in insertion (`seq`) order, recomputing each row's hash from its own stored fields and the *previous row's stored hash* — any row edited in place breaks its own recomputed hash, and (because the next row's `prev_hash` no longer matches) every row after it too. **Verified for real**: created two real audit rows via live HTTP actions, confirmed the CLI (`scripts/verify_audit_log.py`) and the Admin Portal's "Verify integrity" button both report the chain intact — then directly `UPDATE`d one row's `detail` in Postgres (a simulated insider tamper) and confirmed both the CLI (non-zero exit code) and the UI immediately reported the exact broken row and why.
- Wired into a representative, meaningful set of actions (not literally everything in the app — a deliberate scoping choice): admin user create/update, watchlist entry create/update, face-reveal (§3 above), and retention execution (below). Government-registry lookups (M11) have their own separate `RegistryAuditEvent` structured-log trail rather than this table — a documented, not accidental, split.
- **Retention policy engine, the execution half**: `execute_retention` (docs/05-DELIVERY-PLAN.md's M12 "retention policy engine") is the real deletion the M10 preview sliders were always previewing — deletes sealed clips' files from disk *and* rows, and detection rows, older than the configured cutoffs, and records exactly what it deleted (counts, bytes) as its own audited action. The Admin Portal's Retention tab gained a "Delete now" button behind an explicit confirm step.

Honest gaps:

- Not every mutating action in the app is audited — the set above is representative of "sensitive" actions, not exhaustive. Extending coverage to more actions is additive (one `record_audit_event` call each), not a redesign.
- Retention execution is a manual admin action (a button), not a scheduled background job — docs/01-ARCHITECTURE.md's "enforced by a scheduled job, with a visible countdown" is the fuller vision; this ships the enforcement mechanism and its audit trail, not the scheduler.

5. Supply chain: dependency scanning, SBOM, secret scanning

What's real (all reproducible via `make audit` / `make sbom` / `make secrets-scan`, none of it a one-off manual check):

- `pip-audit` against the whole workspace's exported, pinned dependency set (215 packages across every service/package): **no known vulnerabilities.**
- `npm audit` against the frontend's 518 total dependencies: **0 vulnerabilities at every severity level.**
- **CycloneDX SBOMs** for both the Python workspace and the frontend (`make sbom`, using `cyclonedx-bom` / `@cyclonedx/cyclonedx-npm`) — deliberately **not committed** (written to a gitignored `sbom/` directory): an SBOM describes one specific build, and a stale committed one silently describing an old build is worse than no SBOM — the generation capability is what's real and reproducible, not a point-in-time snapshot going stale in git history.
- `detect-secrets` scanned across every source directory (excluding `node_modules` / `.venv` /build caches). Findings reviewed by hand — all 17 hits are expected false positives, not real leaked secrets: Alembic's own auto-generated revision-ID hex strings (flagged as "high entropy," which is exactly what a revision ID is supposed to look like), and documented dev-only default credentials/test fixture values (e.g. `scripts/seed_admin_user.py`'s bootstrap password, `sentinel_core.config`'s already-labelled `"dev-only-*"` defaults) — every one of which this codebase already flags in its own docstrings as insecure-by-design-for-local-dev, per the project's established "compose components at call time from separate fields, never a combined literal" secret-handling convention (see `sentinel_core .config`'s `database_url` / `gov_stream_url` comments).
- Container image scanning (docs/01-ARCHITECTURE.md §"Supply chain: ...container image scanning") is not built — this deployment doesn't build custom container images yet (core_api/edge_agent run as host processes in this dev/demo setup, not containerized).

6. Signed, time-limited media URLs

Decision: HMAC-signed URLs with a short expiry, not unauthenticated media routes. `<video src>` / `<img src>` tags can't attach an `Authorization` header — the standard alternative is a URL whose query string itself proves authorization, for a short, explicit window.

What's real:

- `core_api/security/signed_urls.py`: `sign_media_path()` / `verify_media_signature()` — `HMAC-SHA256(resource_path + ":" + expires_at)`, keyed by a dedicated `media_url_signing_secret` (separate from `jwt_secret`, so rotating one never affects the other). Constant-time comparison (`hmac.compare_digest`) — a signature check that leaks *how much* of the signature matched is a real, if narrow, timing side channel. The signature covers the exact resource path, so a signed URL minted for one detection's snapshot can't be replayed against a different one.
- `core_api/auth/dependencies.py`'s `current_user_or_signed_url` accepts *either* a normal JWT *or* a valid, unexpired `?expires=&signature=` pair — an invalid/expired signature is rejected outright (401) rather than silently falling through to "not authenticated," which would let an attacker distinguish "wrong signature" from "no credential at all."
- Applied to the two media-serving routes that need to work embedded in markup: `GET /api/v1/detections/{event_id}/snapshot` and `GET /api/v1/evidence/clips/{clip_id}/video`. Each has a sibling `.../snapshot-url` / `.../video-url` endpoint (a normal JWT-gated route) that *mints* the signed URL — minting requires a real login; using the minted URL doesn't.
- The clip-video route moved out of `watchlist_router` into its own new `routers/evidence.py`: `watchlist_router` blanket-applies `Depends(current_user)` at `include_router()` time (every route on it requires a JWT, full stop), which would have silently defeated the signed-URL path before `current_user_or_signed_url`'s own logic ever ran. Clip *metadata* (status, sha256, ...) stays on `watchlist_router` unchanged; only the actual video bytes needed the different auth model.
- **Verified live over real HTTP**: minted a signed snapshot URL, fetched it with **no** `Authorization` header at all (200, real image bytes returned); confirmed the same path with no signature and no auth is rejected (401); confirmed a tampered/invalid signature is rejected (401), not silently accepted.

---

7. Stream credentials, published defaults, and what gets audited

Added while preparing for the Phase-2 finale, where the platform meets department cameras and a shared venue network for the first time.

- **Stream credentials never reach an operator.** A camera's RTSP/WHEP URL embeds its account (for the sandbox grid, the team's registered email and access password). `camera_to_out` now masks the userinfo of every profile URL — `rtsp://***@host:port/path` — on every operator-facing endpoint. The only place credentialed URLs leave the API is `GET /api/v1/edge/cameras`, gated by the edge service token, which is what `--source registry` workers read. Asserted in `services/core_api/tests/test_registry_api.py`.
- **Credentials are percent-encoded wherever URLs are built** (`sentinel_core.stream_urls.with_credentials`), so a password containing `@`, `:`, `/`, `#` or `?` cannot re-point a connection at another host.
- **No published secret on a shared network.** The JWT, edge-service, gateway and media-signing secrets default to `dev-only-*` values that are in this repository. Any `SENTINEL_ENVIRONMENT` other than `local` now refuses to start while one of them is unchanged, naming the variables to set. `local` keeps the zero-configuration quickstart, and `make api` binds to localhost only.
- **Audit coverage of evaluation-day actions.** Besides user and watchlist administration, the hash chain now records camera onboarding (manual, bulk CSV, catalogue and device discovery), BOLO arming with its retro-scan result, every alert status change, and each Movement Report export.

Who may do what

Action	Operator	Administrator	Edge worker (service token)
View cameras, map, live wall, search, reports	✓ (own department under RLS, or all if unscoped)	✓	—
Arm a BOLO / add a watchlist entry, triage alerts and zone events	✓	✓	—
Preview a camera CSV (dry run)	✓	✓	—
Add a camera, commit a CSV import, sync a catalogue, discover a device	—	✓	registers its own catalogue cameras only
Configure zone rules, manage users, retention, audit verification	—	✓	—
Post detections, health, zone events, snapshots; read the edge manifest	—	—	✓

The watchlist is statewide by design: a stolen or wanted vehicle is of interest to every department, so watchlist entries are not department-scoped; the alerts they raise are, because those belong to the camera that saw the vehicle.

09 · Secondary Analytics (M13)

Companion to 05-DELIVERY-PLAN.md's M13 milestone: "Vehicle class/colour/coarse-make; zone rules (intrusion, loitering, wrong-way, stopped vehicle); camera tamper detection." Exit bar: "reliable analytics beyond ANPR" — an explicit bonus criterion. Face recognition was excluded by decision (see 00-SOLUTION-PLAN §9); the freed time went into vehicle attributes and zone rules instead, kept honest below against what's actually implemented.

---

1. Vehicle colour classification

Decision: an HSV heuristic, not a trained classifier. Colour classification from a bounding-box crop is a genuinely simpler problem than plate OCR — a small, fixed palette (white/black/silver/grey/red/blue/ green/yellow/other) is separable by saturation/value thresholds and hue ranges without training data or an extra model in the inference budget.

What's real:

- `edge_agent/analytics/colour.py` : `classify_vehicle_colour()` takes the vehicle's cropped BGR pixels, converts to HSV, and buckets by saturation/value (achromatic: white/grey/black by value; chromatic: hue range lookup for red/orange/yellow/green/blue/violet, folded into the fixed palette).
- Wired into `AnprPipeline._classify_colour()` , called once per confirmed vehicle track alongside plate OCR, so it costs nothing extra beyond a crop + `cv2.cvtColor` .
- `Detection.vehicle_colour` column (migration `08e724ff6b3a`), surfaced through `DetectionIn` / `DetectionOut` schemas and `ingest_detection` .
- 11 unit tests over the colour buckets (`test_colour.py`) plus one pipeline-integration test confirming the field actually reaches the emitted detection event.

Honest gap: this is colour only. "Coarse-make" (vehicle manufacturer/ model, e.g. "Maruti Swift" vs "Hyundai i20") is explicitly **out of scope** — it requires a genuinely different trained classifier (fine-grained visual categorisation, not a heuristic), which is a real model-training exercise this session did not undertake. Listed here rather than silently dropped or falsely claimed.

---

2. Camera tamper detection

Decision: frame statistics, not a trained model. The three tamper modes worth detecting in practice — a lens covered, a lens smeared/ defocused, a camera physically knocked or panned — all show up as gross, easily-computed statistical shifts in the raw frame, not subtle patterns needing a classifier.

What's real:

- `edge_agent/analytics/tamper.py` 's `TamperDetector` : maintains a rolling reference frame and flags:
  - **covered** — near-uniform pixel intensity (low overall variance) sustained across consecutive frames.
  - **blurred** — sustained low Laplacian variance (the standard focus-measure operator; a sharp frame has high-frequency edges, a defocused one doesn't).
  - **moved** — a large frame-to-frame greyscale-histogram shift, indicating the field of view itself changed (pan/knock), as opposed to normal scene motion within a stable frame.
  - Each mode requires *sustained* evidence across several frames before flagging, specifically to avoid false positives on a single dark frame, a fast-moving foreground object, or a momentary glare.
- Runs inside the existing per-camera health-reporting loop in `worker.py` 's `_report_health` / `_run_camera` — no separate polling path, no extra model download.
- `Camera.tamper_status` column (migration `3a9af9c7bca1`), surfaced through `CameraHealthUpdate` / `CameraOut` schemas and `update_camera_health` / `camera_to_out` .
- Frontend: `Cameras.tsx` gained a **Tamper** column; `SeverityBadge.tsx` 's new `tamperToSeverity()` maps `ok` → neutral, `covered` / `blurred` / `moved` → warning-styled badges.
- 7 unit tests (`test_tamper.py`) plus 2 new `test_registry_service.py` tests covering the health-update wiring.
- **Verified live in-browser**: screenshot-confirmed a styled "covered" badge rendering correctly in the Cameras table against a real camera reporting real tamper state.

---

3. Zone rules engine

Decision: hand-rolled polygon geometry over normalised coordinates, no new geometry dependency. Zones are simple, mostly-convex regions (a lane, a gate, a loading bay); ray-casting point-in-polygon is a dozen lines and avoids pulling in `shapely` for a problem this small. Coordinates are normalised to `[0, 1]` per axis so a zone definition is resolution- independent — the same zone works whether the camera streams at 640×480 or 1920×1080.

What's real:

- `edge_agent/analytics/zone_rules.py`: `Zone` (polygon + rule type + thresholds), `ZoneEvent` (fired violation), `ZoneRuleEngine` (stateful, keyed per tracked-vehicle-id):
  - **intrusion** — fires once when a track's position enters the polygon; resets when it exits, so re-entry fires again.
  - **loitering** / **stopped_vehicle** — fire once a track's dwell time inside the zone (loitering: any movement; stopped_vehicle: near-zero speed specifically) exceeds a configurable threshold, tracked via a per-track `fired_rules` set so a single continuous dwell fires once, not once per frame; cleared on exit or resumed movement.
  - **wrong_way** — fires once a track's heading sustains a deviation beyond tolerance from the zone's configured expected direction; cleared when heading returns within tolerance.
  - A `_TrackState.positions` rolling window (default 5 s) backs both the `speed()` and `heading_deg()` computations used by the above.
- Backend: `Zone` / `ZoneEvent` ORM models with `FORCE ROW LEVEL SECURITY` policies matching M12's department-tenancy pattern (migration `6943898d1be1`); `core_api/zones/` (schemas + service) and `core_api/routers/zones.py`, registered in `app.py`.
- Edge wiring: `worker.py`'s `_load_zone_engine()` fetches a camera's configured zones over a real HTTP call at startup — the same "build it like a real integration, default to nothing configured" pattern used for M11's government-registry providers, rather than hardcoding zones into the worker. `_evaluate_zones()` runs the engine against `AnprPipeline.last_tracked_vehicles` each frame and POSTs any fired events back to core.
- Frontend: `CameraDetailModal.tsx`'s new `ZoneRulesSection` — a preset-region picker (full frame / left / right / top / bottom / centre third, since free-hand polygon drawing was out of scope for the time available), a rule-type selector, a zone list, and a recent-events list. `types.ts` / `api.ts` gained the corresponding `Zone` / `ZoneCreate` / `ZoneEvent` types and `zonesApi` client.
- 12 unit tests (`test_zone_rules.py`, covering all four rule types plus fire/reset semantics) and 7 `test_zones_service.py` tests.
- **Verified live in-browser, end to end**: opened a real camera's detail modal against the running dev stack, created a "Highway Shoulder" / `stopped_vehicle` zone through the UI, and confirmed it rendered correctly in the zone list — against a live synthetic camera feed actually running through the edge worker.

Honest gap: zone *shape* is limited to six coarse presets, not free-hand polygon drawing on the video canvas — the underlying engine supports arbitrary polygons (it's just point-in-polygon over a coordinate list), but a drag-to-draw canvas editor was descoped as a UI investment not justified by the time remaining relative to its marginal value over the presets.

3a. People: counting and zone intrusion, never identity

The official evaluation lists person detection next to vehicle detection. The vehicle detector's COCO model already finds people, so the same single inference now reports them (`DETECTED_CLASSES` in `vehicle_detector.py`):

- **Counting.** Each heartbeat carries the most people and vehicles seen in one analysed frame since the last (`people_in_view`, `vehicles_in_view`). The Cameras screen shows them under the ANPR rate: crowd and queue counting with no extra model.
- **Zones.** People are tracked on their own tracker and held to intrusion and loitering zones (a person standing still is not a stopped vehicle, and has no wrong way). Their zone events are labelled `person` in the queue.
- **No identity.** People are never read, matched or followed across cameras. Faces in evidence stay blurred. This keeps the no-face-recognition position.
- **Vehicles are unaffected.** People get their own non-maximum suppression, so a rider's box can never suppress the motorcycle under them. On 120 real government-grid frames: 185 people found in 90 frames, and zero vehicle boxes lost against vehicle-only detection.

---

4. Verification summary

- Full backend suite at the time of this milestone: **353 tests passing** (up from 298 at the end of M12; 359 after the M14/M15 work), `mypy` and `ruff` clean across `packages`, `services`, `tests`.

- Frontend: `npm run build` (the authoritative `tsc -b` + Vite build) clean.
- All three features additionally verified against the *live* dev stack (real Postgres, a real edge worker driving 8 synthetic cameras, real browser interaction) — not unit tests alone standing in for integration proof.

12 · Requirements traceability

Every requirement on sentinel.gujarat.gov.in, where it is met, and the evidence

Checked against the official problem statement, solution framework, evaluation framework and FAQ on the hackathon site. **Status** is one of *Met*, *Partial* (stated limit in the row) or *Not built*. Evidence links point at code, a test or a measured run, never only at a design document.

---

1. Common evaluation areas

#	Area	Status	Where	Evidence
01	Successful test case: onboard the government feed, view it, produce analytics output	**Met**	Registry import / catalogue onboarding, relay viewing, registry-driven worker	`M4 gov-grid integration`, `M14 evidence run`, runbook §5
01	Track the designated vehicle across cameras and times	**Met**	Find a Vehicle → route, movement report (PDF/CSV/ZIP with hashes)	`Cross-camera findings`, runbook §7
02	Solution presentation	**Met**	`make deck` → `docs/Sentinel-Platform-Deck.pptx`	Submission playbook
03	Solution architecture (HLD, diagrams, security, interoperability)	**Met**	`docs/Sentinel-Platform-HLD.pdf`, 01-ARCHITECTURE, 08-SECURITY	Workflow diagram PDF in `docs/`
04	Working platform on own feed and government feed	**Met**	Operator console, field PWA, edge worker	`M15 fresh-clone test`, `chaos drills`
05	ANPR	**Met, accuracy stated**	Detect → track → plate → OCR → grammar correction → vote	`ANPR accuracy`: 65% exact per crop, 78% after voting
05	Vehicle detection and classification	**Met**	YOLO11n, car / motorcycle / bus / truck, colour	`edge_agent/analytics/vehicle_detector.py`
05	Person detection	**Met**	Same inference as vehicles; people counted per camera and held to intrusion/loitering zones; never identified	09-SECONDARY-ANALYTICS §3a
05	Intrusion detection	**Met**	Zone rules: intrusion, loitering, wrong-way, stopped vehicle; triaged as alerts	`edge_agent/analytics/zone_rules.py`, Zone event queue
05	Timestamps	**Met**	Camera's burned-in clock preferred, PTS otherwise; IST everywhere in the UI and reports	`frame_clock.py`, `apps/web/src/lib/time.ts`
05	Output reports	**Met**	Movement report PDF with route sketch and evidence frames; CSV; evidence ZIP with SHA-256 manifest; `scripts/evidence_report.py`	`core_api/reports/service.py`
06	Scalability toward ~80,000 cameras	**Met (plan + measurement)**	Tiered analytics, federation, edge processing	04-SCALE-PLAN, §4 below
06	PoC readiness	**Met**	Finale runbook, DR drill, `make doctor` / `make prefetch`	10-FINALE-RUNBOOK, `DR drill`
07	Submission completeness	**Met**	Checklist and packaging	06-SUBMISSION-PLAYBOOK

2. Bonus considerations

Bonus	Status	What is real
Innovative hybrid architecture	**Met**	Model 1 + 2 + 3 + selective 4 (00-SOLUTION-PLAN §2)
Advanced cross-camera tracking / multi-camera correlation	**Met**	Matching ladder (exact, OCR-ambiguity, one-character, bridged unreadable hops), feasibility gate on implied speed
Additional reliable analytics beyond ANPR	**Met**	Colour, tamper, zone rules, people and vehicle counts, coverage-gap and lifecycle analysis
Edge processing, bandwidth optimisation, low connectivity	**Met**	Edge worker sends metadata only (plus snapshots of detections), tiered analysis, a durable outbox that replays detections after an outage
Cybersecurity, privacy, auditability, RBAC	**Met**	mTLS, RBAC/ABAC, Postgres RLS, hash-chained audit log (tamper detected, survives restore), face blur, admin-only registry writes, private-camera consent enforcement
Dashboards, automated alerts, health monitoring, APIs	**Met**	Server-sent-event alerts with sound, camera health (live / stale / degraded / down, analysed fps, lag), OpenAPI

3. HLD content the FAQ requires

Required in the HLD	Where
Overall architecture with diagrams	01-ARCHITECTURE, HLD PDF
Heterogeneous cameras / VMS (IP, analog via NVR, multi-vendor, mixed protocols)	07-DRIVER-SDK: RTSP, ONVIF, HLS, Hikvision ISAPI, Dahua CGI, Genetec and Milestone VMS
Geographically dispersed sites (bandwidth, connectivity, edge vs central)	04-SCALE-PLAN §2
Video analytics approach (ANPR, cross-camera tracking)	02-ANPR-PIPELINE, 09-SECONDARY-ANALYTICS
Scalability for ~80,000 cameras	04-SCALE-PLAN
Technical details needed from departments	04-SCALE-PLAN §8, runbook §1

4. Scale: measured versus projected

Quantity	Measured (where)	Projected at 80,000 cameras
Analytics per laptop, dense traffic (a plated vehicle in every frame)	Base M2: 2 cameras at 8 fps keep up; 4 fall behind (`tier capacity`)	Sized per GPU in 04 §3; tiering means only 12% of cameras run continuous ANPR
Plate-read ceiling	~30 reads/s on M2 CPU	GPU inference in production
Restore of 1 M detections	5.7 s (`DR drill`)	RTO via replica failover, not dump/restore
ANPR exact match	65% per crop, 78% voted, one camera (`accuracy`)	Fine-tuning on Indian plates first
80,000 cameras deployed	Not deployed	Arithmetic in 04-SCALE-PLAN

5. Government database integration readiness

The brief names five systems. Each has a provider behind one `ExternalRegistry` port, with retry, circuit breaker, rate limit and a redacted audit record, and is checked live from **Admin → Integrations**. No production credential exists, so each runs against a contract-shaped mock; on the day access is granted it is a configuration change.

System	Owner	Provider	Operations	A rejected credential
VAHAN	MoRTH	`VahanProvider`	Vehicle registration lookup	Raises `RegistryAuthError`
SARTHI	MoRTH	`SarthiProvider`	Driving licence lookup	Raises
eGujCop (CCTNS)	Gujarat Police	`EGujCopProvider`	Person / case lookup, alert push	Raises
AFIS	Gujarat Police	`AfisProvider`	Biometric identity match	Raises
NAFIS	NCRB	`NafisProvider`	National biometric match, cross-state	Raises

A credential failure never reads as "no record": an expired key must not make a wanted vehicle look clean.

6. Known gaps, stated

Requirement	Position
Face recognition (FRS)	**Excluded by decision.** Faces are detected only to be blurred. The system answers "where did this vehicle go", not "where did this person go" (00-SOLUTION-PLAN §6).
Coarse make / model classification	Not built; needs a trained classifier.
Real Hikvision / Dahua hardware	Drivers are written to the vendors' published APIs and tested against mock servers that implement them, including digest auth; no physical device has been on the bench. ONVIF and plain RTSP reach the same devices. See 07-DRIVER-SDK.

7. Private cameras

The FAQ asks for viewing of public-facing private cameras (societies, malls, shops) "wherever feasible and permitted". Permission is enforced, not assumed:

- A private camera is onboarded with `ownership=private`, the consent reference it was shared under (`consent_ref`) and when that consent ends (`consent_expires_on`).
- Without both, or after the date, the camera stays in the registry but is **not served to viewers and not analysed**: the worker manifest leaves it out and the stream endpoint refuses it with the reason.
- **Coverage & lifecycle** counts lapsed and expiring consents per department, so the owning station renews them before they lapse.
- Private cameras default to view-only (`d_registry_only`); analysing one is an explicit tier choice.